

Neural networks for sequential (and other) data

12th March, 2026



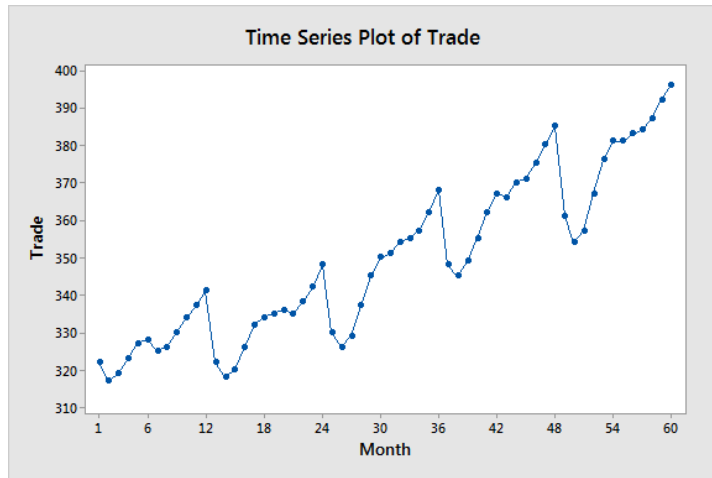
Alexey Zaytsev,

Assistant professor, Skoltech

Classic ML for time series data

Follow a **simple recipe**:

1. Take input data including history for the target variable
2. Add differences, combinations, rolling means, medians, etc.
3. Add one-hot-encoding for important categorical features like day of week, holiday indicator etc.

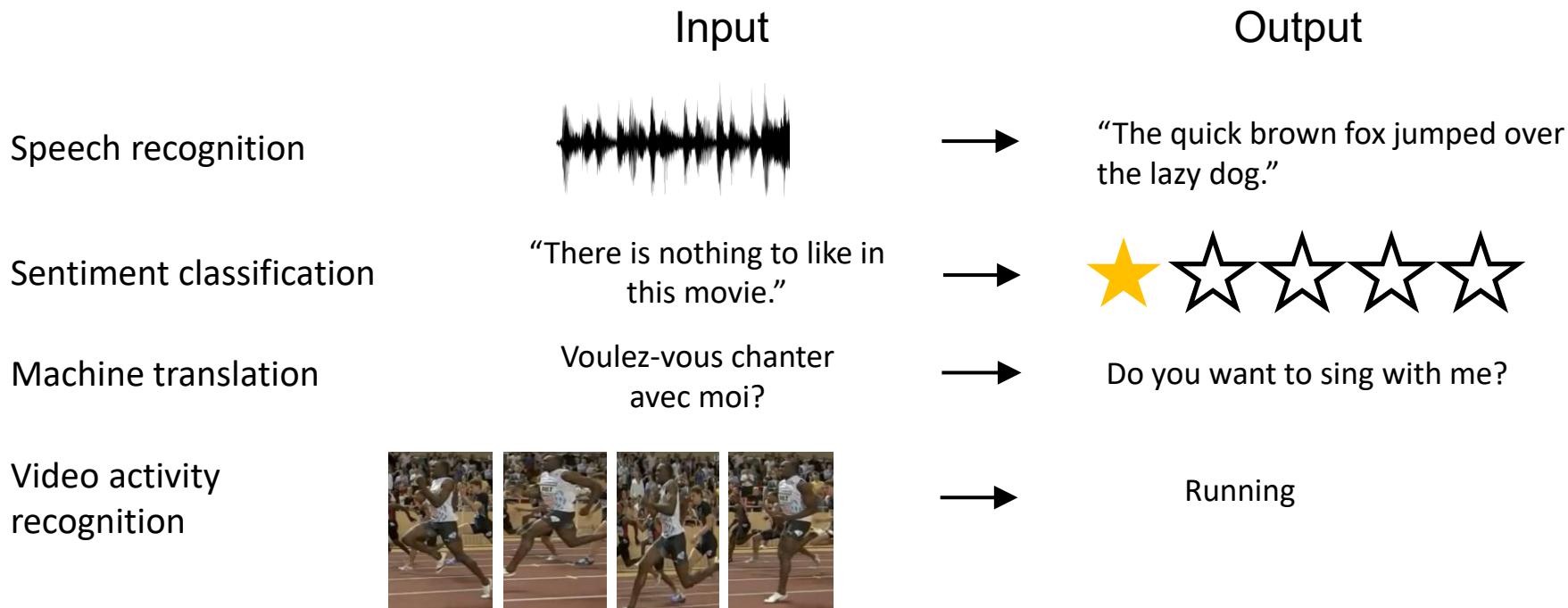


Time series example: let us forecast

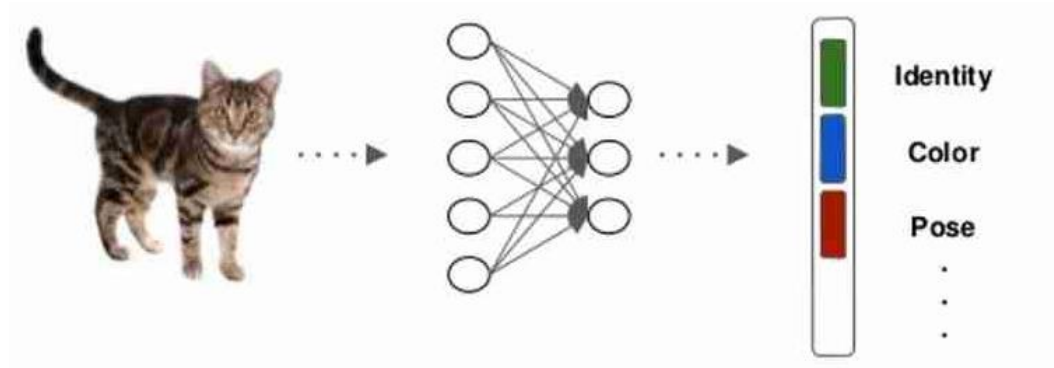
- Now we have input features for all points
- Let's apply our favorite ML regression algorithm

dmlc
XGBoost

Deep Learning problems with sequential data: we need representations



Semi-structured data processing: why do we need NNs?



Fundamental example: a next word prediction

If we can predict the next word, we can answer any question by adding words one by one

The most complicated and difficult part of it was only just beginning.

What we have

What we want to predict

Naïve approaches to the next word prediction

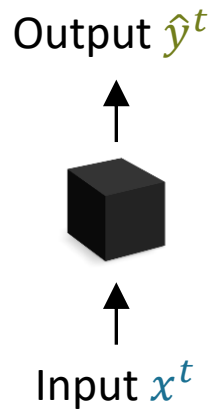
Idea 1: use previous word(s)

Problem 1: long-term dependencies

“**France** is where I grew up, but I now live in Boston. I speak fluent ____.”

The most complicated and difficult part of it was only just beginning.

Feature representation: [0, 0, 0, 1, 0, 0]



Naïve approaches to the next word prediction

Idea 1: use previous word(s)

Problem 1: long-term dependencies

Idea 2: use bag of words model

The most complicated and difficult part of it was only just beginning.



Feature representation: [0, 3, 0, 2, 0, 0]

Bag of words: number of occurrences of each word

Naïve approaches to the next word prediction

Idea 1: use previous word(s)

Idea 2: use bag of words model

Problem 1: long-term dependencies

Problem 2: order preservation

The food was good, not bad at all.

vs.

The food was bad, not good at all.

The most complicated and difficult part of it was only just beginning.



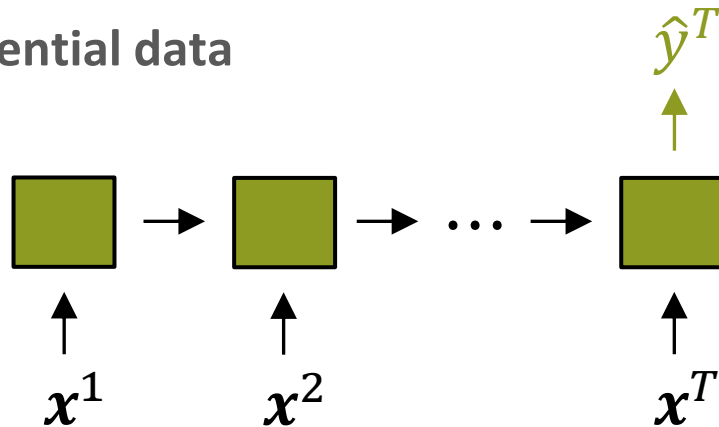
Feature representation: [0, 3, 0, 2, 0, 0]

Bag of words: number of occurrences of each word
(see also TF-IDF features)

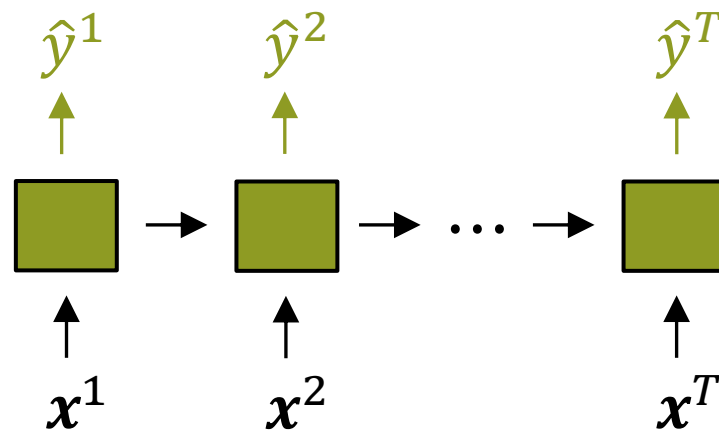
Problem statements for sequential data



Sequence to
a single output

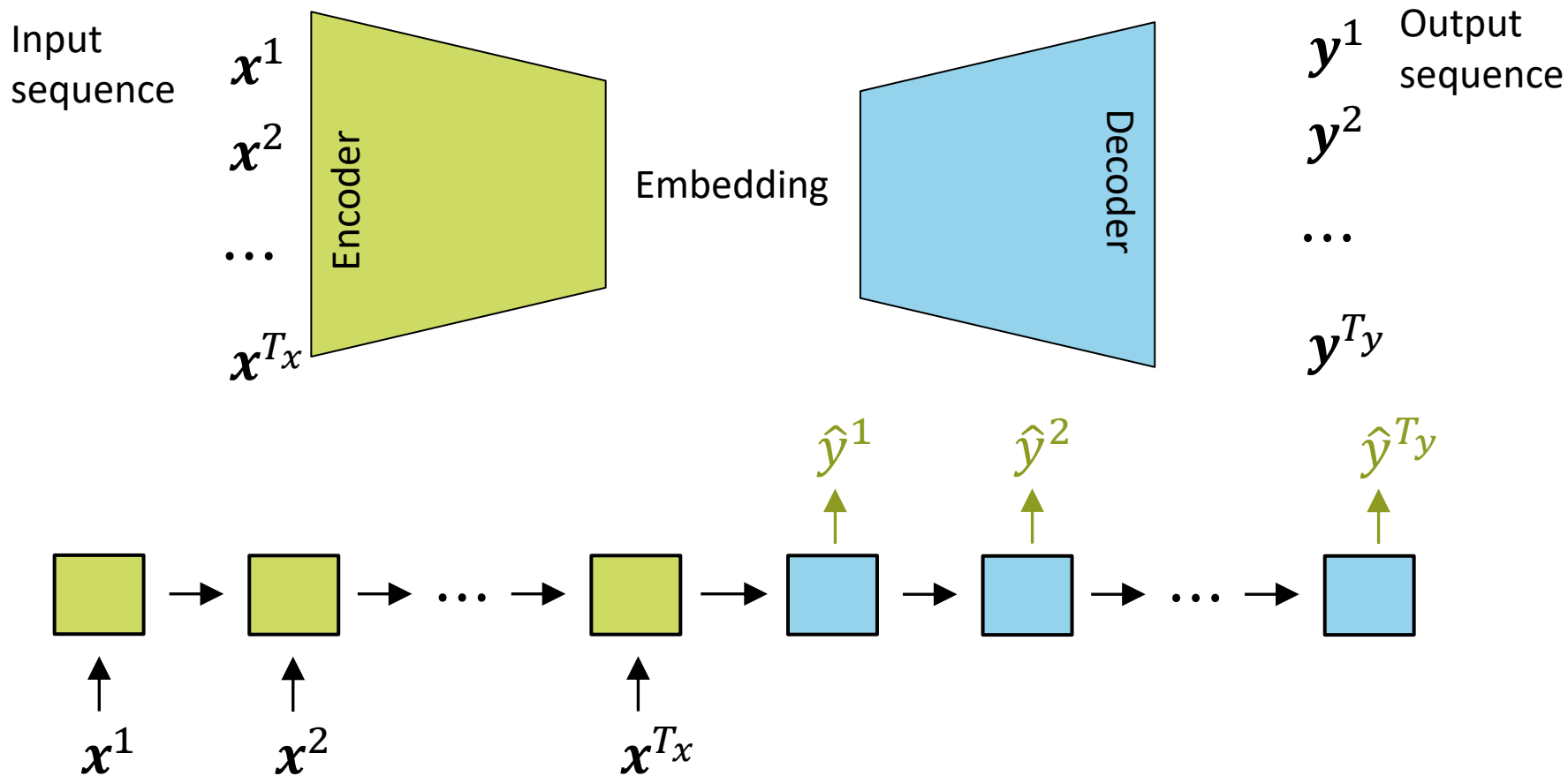


Sequence to
a sequence of
outputs



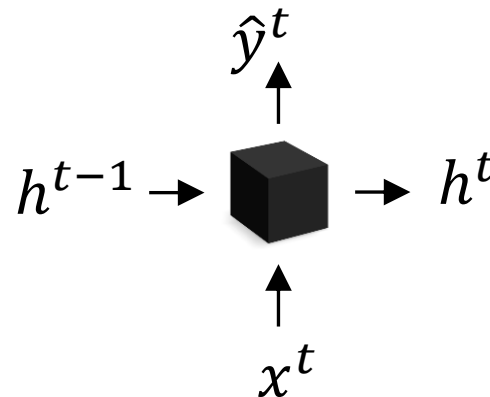
No recurrent
structure

A sequence-to-sequence problem



Model Design Criteria

1. Variable-length sequences processing
2. Long-term memory
3. Maintain order information
4. Natural preprocessing



Blackbox **Neural Networks** are the solution!

Sequence processing with classic ML models

generated features + *dmlc*
XGBoost

1. Variable-length sequences processing

YES (if one to one)

2. Long-term memory

NO



3. Maintain order information

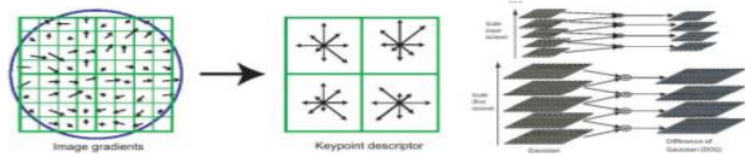
NO



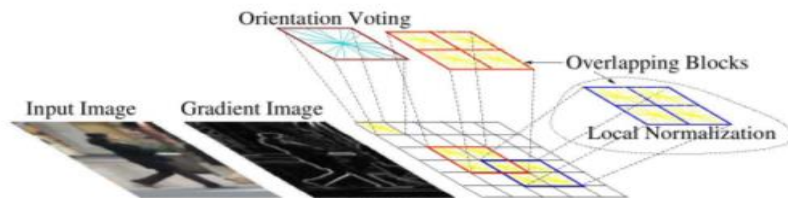
4. Natural preprocessing

a kind of

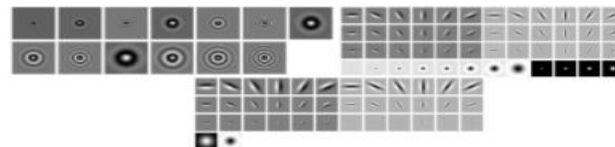
An art of feature construction



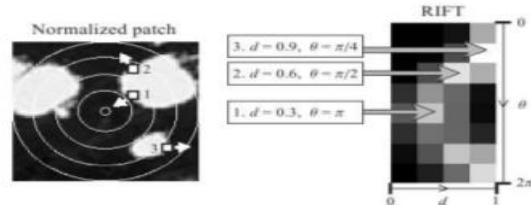
SIFT



HoG

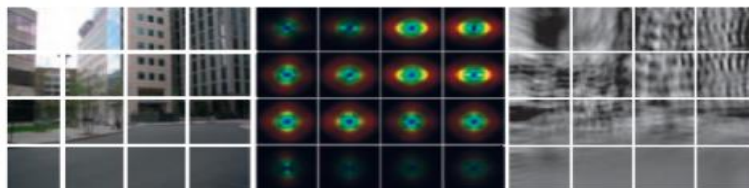


Textons

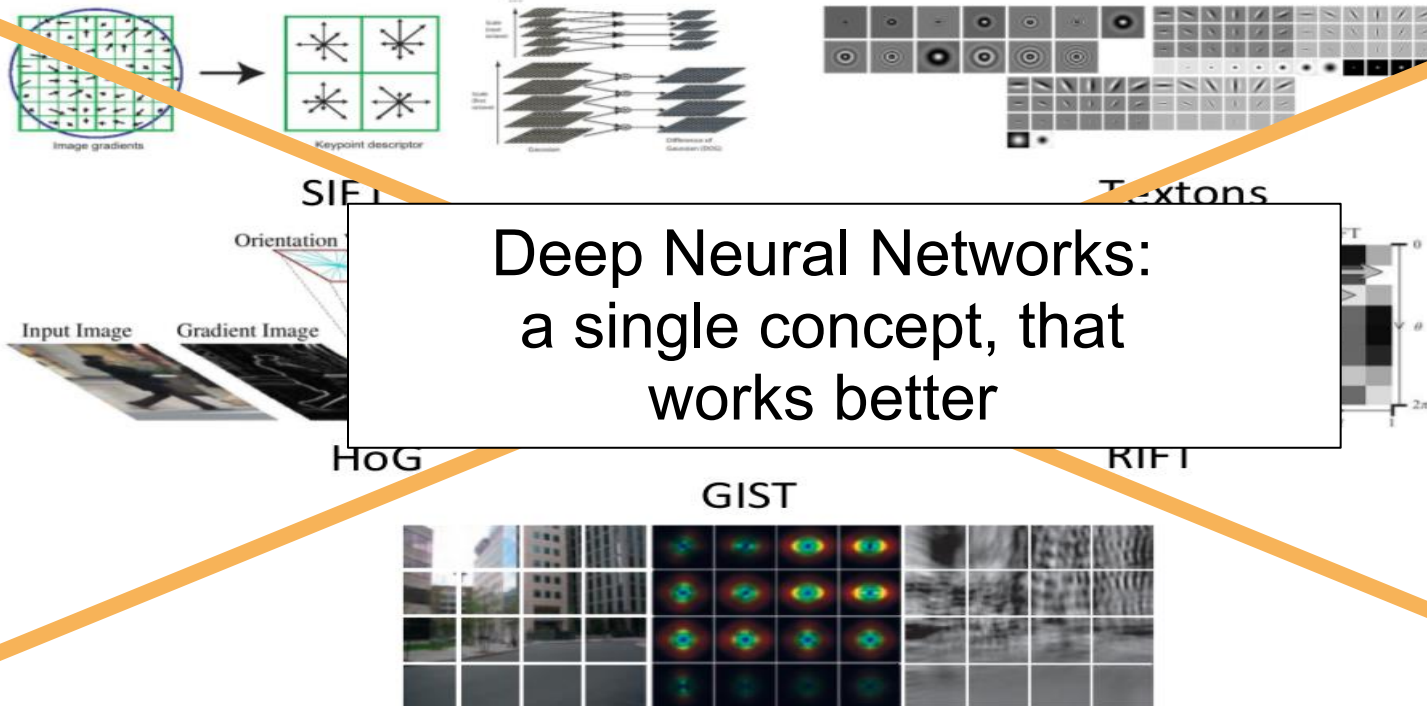


RIFT

GIST



No art now, just engineering



Sequence data examples

Speech recognition



Sentiment classification

“There is nothing to like in
this movie.”

DNA sequence analysis

AGCCCCTGTGAGGAACTAG

Machine translation

Voulez-vous chanter
avec moi?

Video activity
recognition



Large data set
for training

Semi-
structured data

+

+

+

+

+

+

+

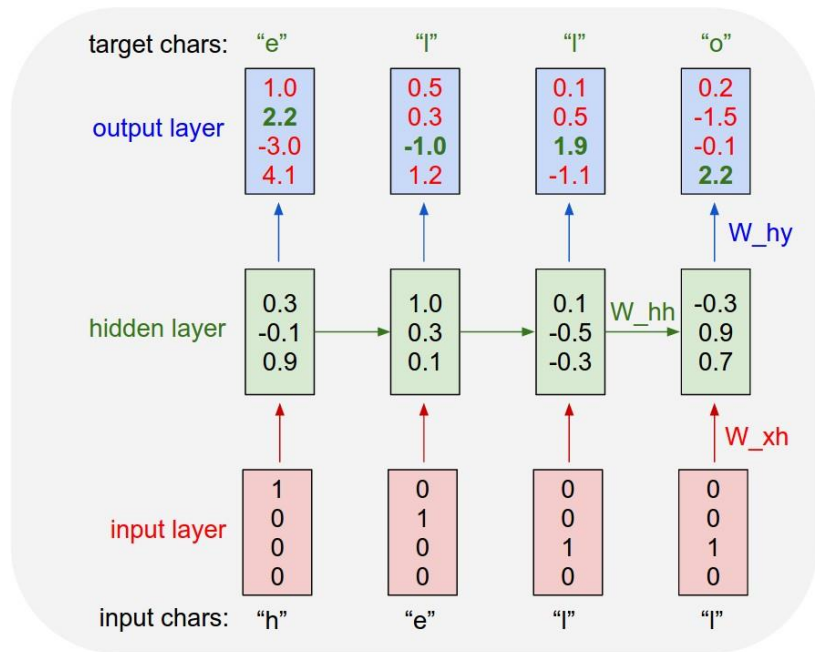
+

+

+

Examples of texts generated by LSTM (Long Short Term Memory NN)

- Shakespeare
- Wiki
- Algebraic geometrics articles
- Linux Source Code
- Dinosaurs' names



For $\bigoplus_{i=1, \dots, n} \mathcal{L}_{\mathcal{O}_X}$ where $\mathcal{L}_{\mathcal{O}_X} = 0$, hence we can find a closed subset $\mathcal{H}$ in $\mathcal{H}$ and any sets $\mathcal{F}$ on X , U is a closed immersion of S , then $U \rightarrow T$ is a separated algebraic space.

Proof. Proof of (1). It also start we get

$$S = \text{Spec}(R) = U \times_X U \times_X U$$

and the comparably in the fibre product covering we have to prove the lemma generated by $\coprod Z \times_U U \rightarrow V$. Consider the maps M along the set of points Sch_{aff} and $U \rightarrow U$ is the fibre category of S in U in Section. 77 and the fact that any U affine, see Morphisms, Lemma 77. Hence we obtain a scheme S and any open subset $W \subset U$ in $\text{Sch}(G)$ such that $\text{Spec}(R) \rightarrow S$ is smooth or an

$$U = \bigcup U_i \times_S U_i$$

which has a nonzero morphism we may assume that f_i is of finite presentation over S . We claim that $\mathcal{O}_{X, \mathcal{O}_X}$ is a scheme where $x, x', x'' \in S'$ such that $\mathcal{O}_{X, \mathcal{O}_X} \rightarrow \mathcal{O}_{X, \mathcal{O}_X}$ is separated. By Algebra, Lemma 77 we can define a map of complexes $\text{GL}_2(x'/S')$ and we win. $\square$

To prove study we see that $\mathcal{F}_{U_i}$ is a covering of X' , and $\mathcal{T}_i$ is an object of $\mathcal{F}_{X'/S}$ for $i > 0$ and $\mathcal{F}_0$ exists and let $\mathcal{F}_i$ be a presheaf of $\mathcal{O}_X$ -modules on $\mathcal{C}$ as a $\mathcal{F}$ -module. In particular $\mathcal{F} = U/\mathcal{F}$ we have to show that

$$\hat{\mathcal{F}}^* = \mathcal{I}^* \otimes_{\mathcal{O}_{\mathcal{F}(U)}} \mathcal{O}_{\mathcal{F}} \sim \mathcal{I}^* \otimes \mathcal{F}$$

is a unique morphism of algebraic stacks. Note that

$$\text{Arrows} = (\text{Sch}/S)_{\text{aff}}^{\text{op}} \times_{\text{Sch}(S)} (\text{Sch}/S)_{\text{aff}}$$

and

$$V = \Gamma(S, \mathcal{O}) \rightarrow (U, \text{Spec}(A))$$

is an open subset of X . Thus U is affine. This is a continuous map of X is the inverse, the groupoid scheme S .

Proof. See discussion of sheaves of sets. $\square$

The result for prove any open covering follows from the less of Example 77. It may replace S by $X_{\text{étale}}$, which gives an open subspace of X and T equal to $S_{Z_{\text{étale}}}$ see Descent, Lemma 77. Namely, by Lemma 77 we see that R is geometrically regular over S .

Lemma 0.1. Assume (1) and (2) by the construction in the description.

Suppose $X = \text{lim}[X]$ (by the formal open covering X and a single map $\text{Proj}_X(A) = \text{Spec}(B)$ over U compatible with the complex

$$\text{Set}(A) = \Gamma(X, \mathcal{O}_X \otimes_{\mathcal{O}_X} \mathcal{O}_X)$$

When in this case of to show that $Q \rightarrow Q_{2, X}$ is stable under the following result in the second conditions of (1), and (2). This finishes the proof. By Definition 77 (without element is when the closed subschemes are categorical. If T is surjective we may assume that T is connected with residue fields of S . Moreover there exists a closed subspace $Z \subset X$ of X where U is proper (some defining as a closed subset of the uniqueness it suffices to check the fact that the following theorem (1) I is locally of finite type. Since $S = \text{Spec}(R)$ and $Y = \text{Spec}(R)$.

Proof. This is form all sheaves of sheaves on X . But given a scheme U and a surjective étale morphism $U \rightarrow X$. Let $U \cap U = \coprod_{i=1, \dots, n} U_i$ be the scheme X over S at the schemes $X_i \rightarrow X$ and $U = \text{lim}_i X_i$. $\square$

The following lemma surjective restrocomposes of this implies that $\mathcal{F}_{\mathcal{O}_X} = \mathcal{F}_{\mathcal{O}_X} = \mathcal{F}_{\mathcal{O}_X}$.

Lemma 0.2. Let X be a locally Noetherian scheme over S , $E = \mathcal{F}_{X/S}$. Set $Z = \mathcal{F}_1 \subset \mathcal{L}_n$. Since $\mathcal{T}^* \subset \mathcal{T}^*$ are nonzero over $\mathcal{O}_S \leq \mathcal{P}$ is a subset of $\mathcal{F}_{\mathcal{O}_S} \circ \mathcal{A}_2$ works.

Lemma 0.3. In Situation 77. Hence we may assume $\mathfrak{q} = 0$.

Proof. We will use the property we see that $\mathfrak{p}$ is the next functor (77). On the other hand, by Lemma 77 we see that

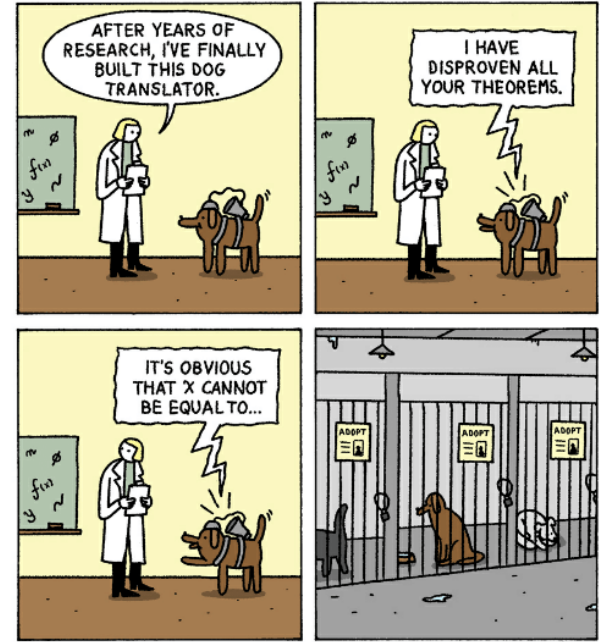
$$D(\mathcal{O}_X) \simeq \mathcal{O}_X(D)$$

where K is an F -algebra where δ_{n+1} is a scheme over S . $\square$

Conclusions

- Classic ML can't handle semi-structured data common in sequential data processing
- We can *learn representations* via Neural Networks
- Results for NNs are nice even for relatively simple and old models

Machine translation: application example



War and Pao

Dog Translation Machine

Machine translation, 50-s

Cold war child: translator from Russian to English IBM 701 Translator

Doctor Dostert predicted that “five, perhaps three years hence, interlingual meaning conversion by electronic process in important functional areas of several languages may well be an accomplished fact.” (1954)

Rule-based approach that uses English-Russian dictionary

Project shut-down in 5 years: no significant progress

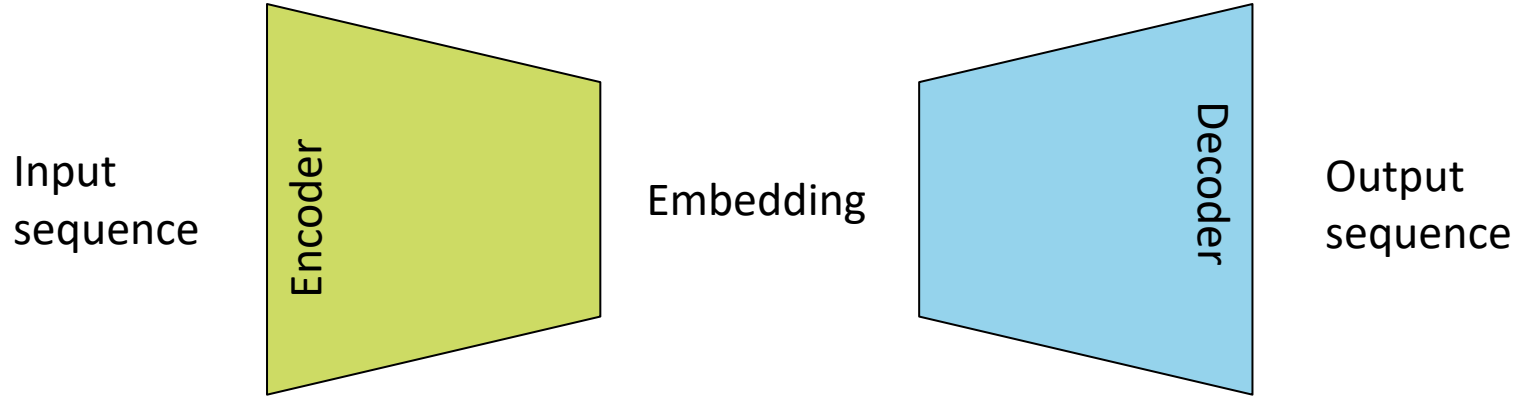


https://www.ibm.com/ibm/history/exhibits/701/701_translator.html
<https://youtu.be/8ZtdVUB007A>

Statistical approach – the leading one before 2014

- Complicated and heavy model
- Many separate components
- Complex generation of inputs
- Support of a sophisticated system
- Quality is not great

Neural network for machine translation



seq2seq (sequence to sequence) architecture

Vanilla Recurrent Neural Network



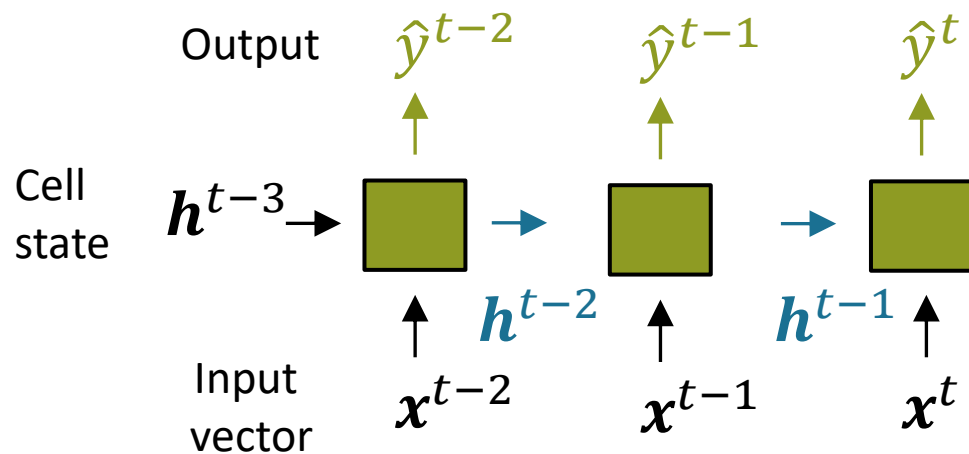
Simple RNN model

At each step:

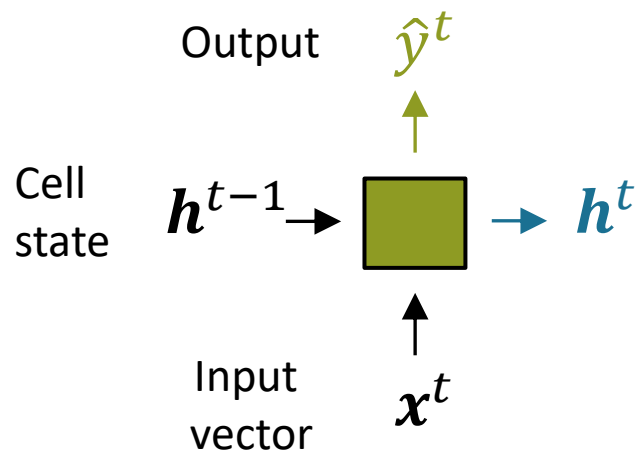
$\hat{y}^t$ - output / model prediction

x^t - input vector / new information

h^t - cell / hidden state



Simple RNN model block



$$\mathbf{h}^t = f_h(\mathbf{x}^t, \mathbf{h}^{t-1})$$

$$\mathbf{h}^t = \tanh(V\mathbf{x}^t + W\mathbf{h}^{t-1} + b_h)$$

$$\hat{\mathbf{y}}^t = f_y(\mathbf{h}^t)$$

$$\hat{\mathbf{y}}^t = \text{softmax}(U\mathbf{h}^t + b_y)$$

Sequence processing with Vanilla RNN

- Long-term memory
- Maintain order information
- Natural preprocessing
- Variable-length sequences processing

NO

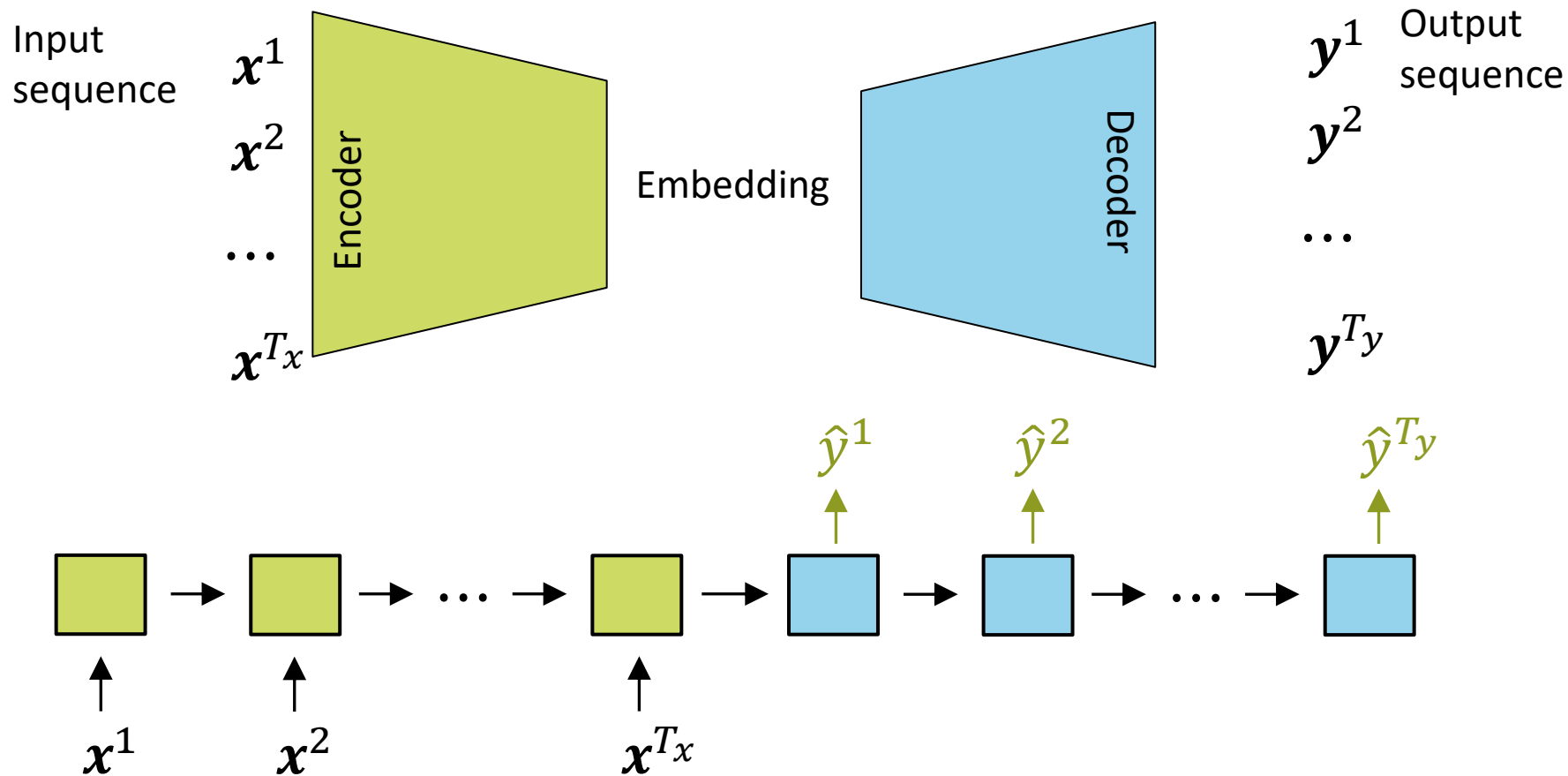


YES

a kind of

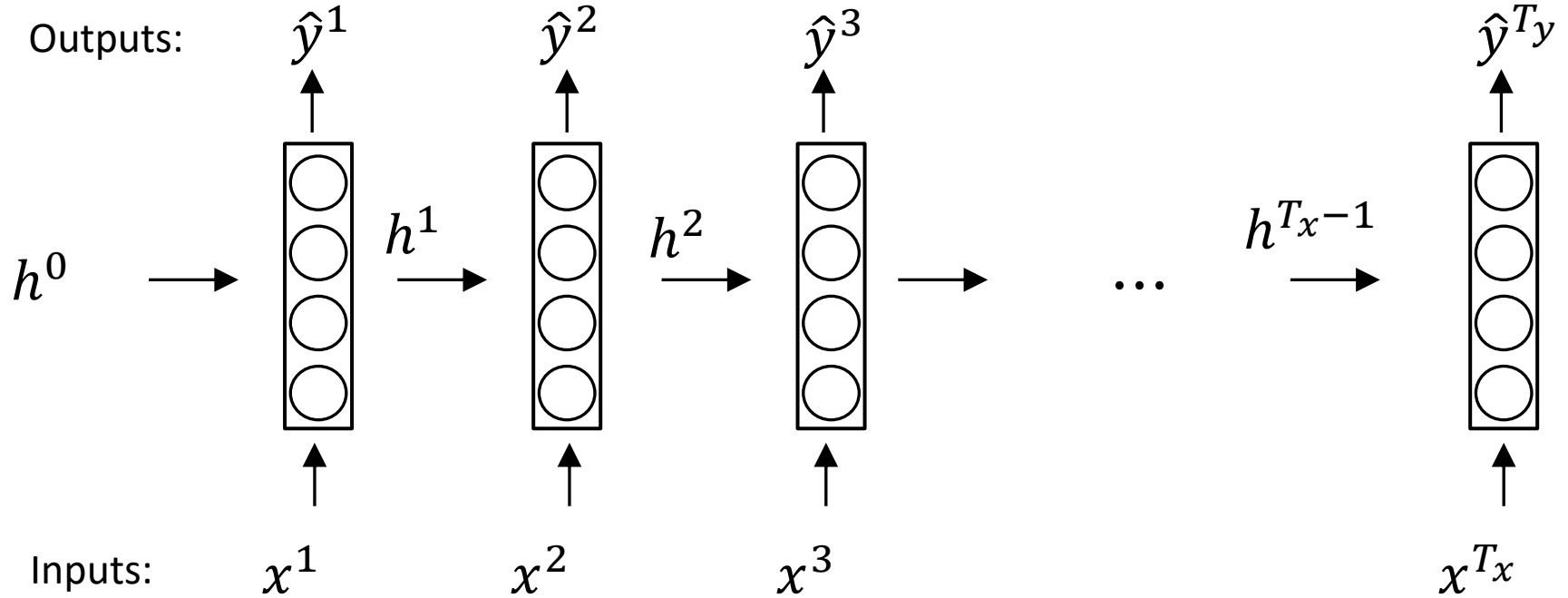
YES (if one to one)

A sequence-to-sequence problem

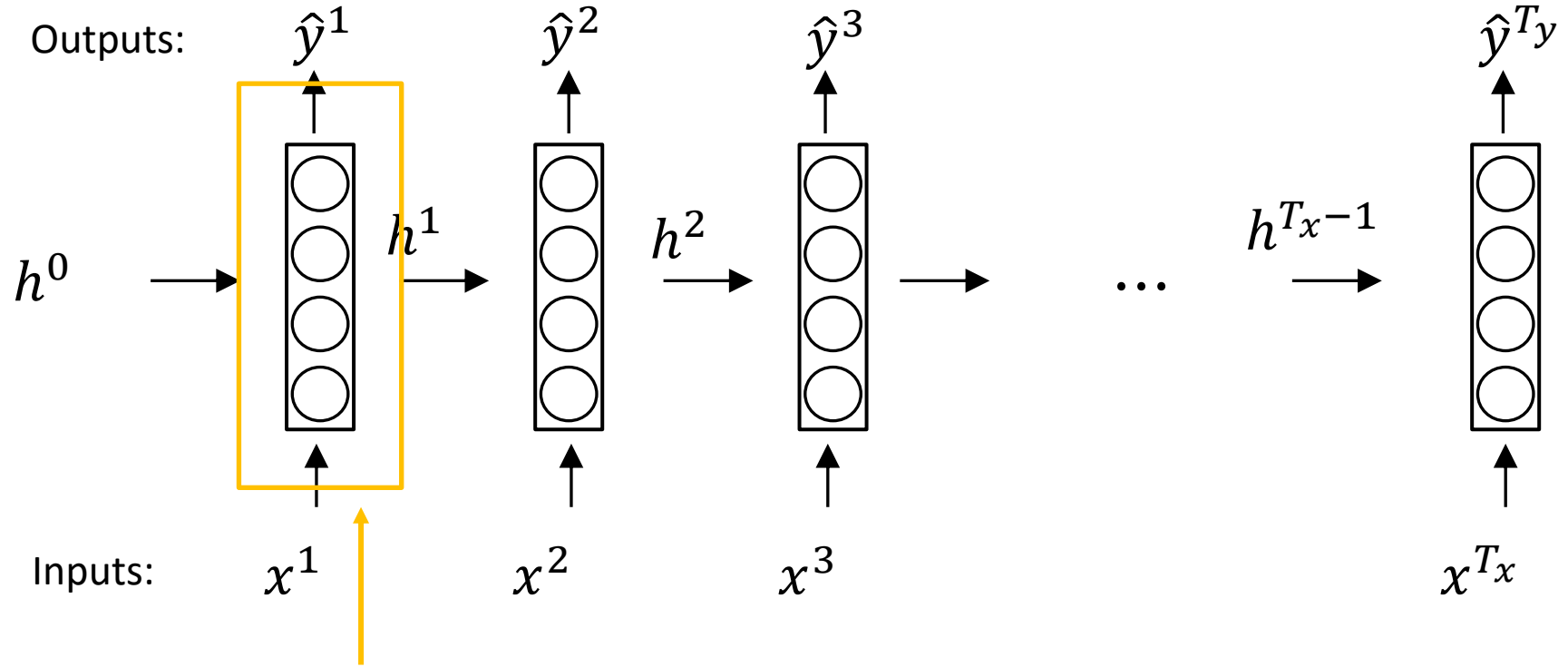


More on Recurrent Neural Networks

Forward propagation through Recurrent Neural Network

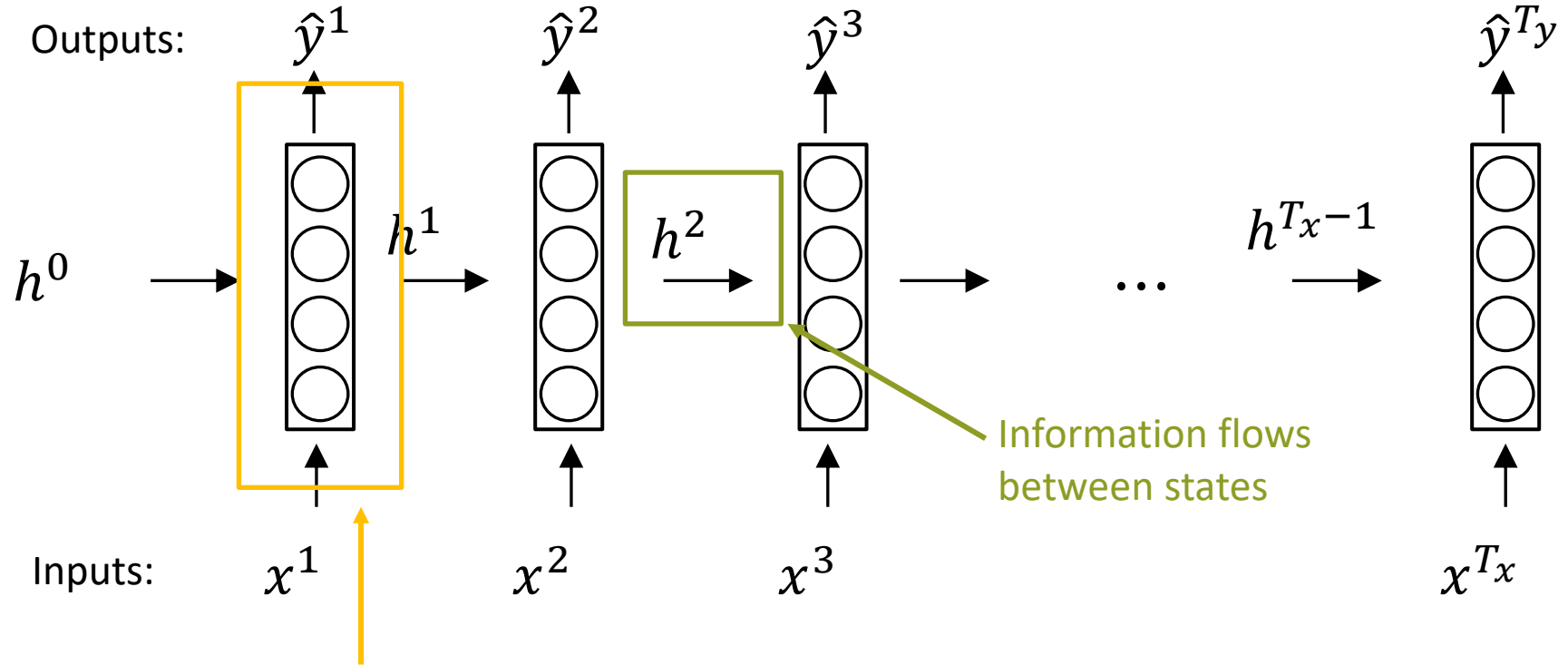


Forward propagation through Recurrent Neural Network



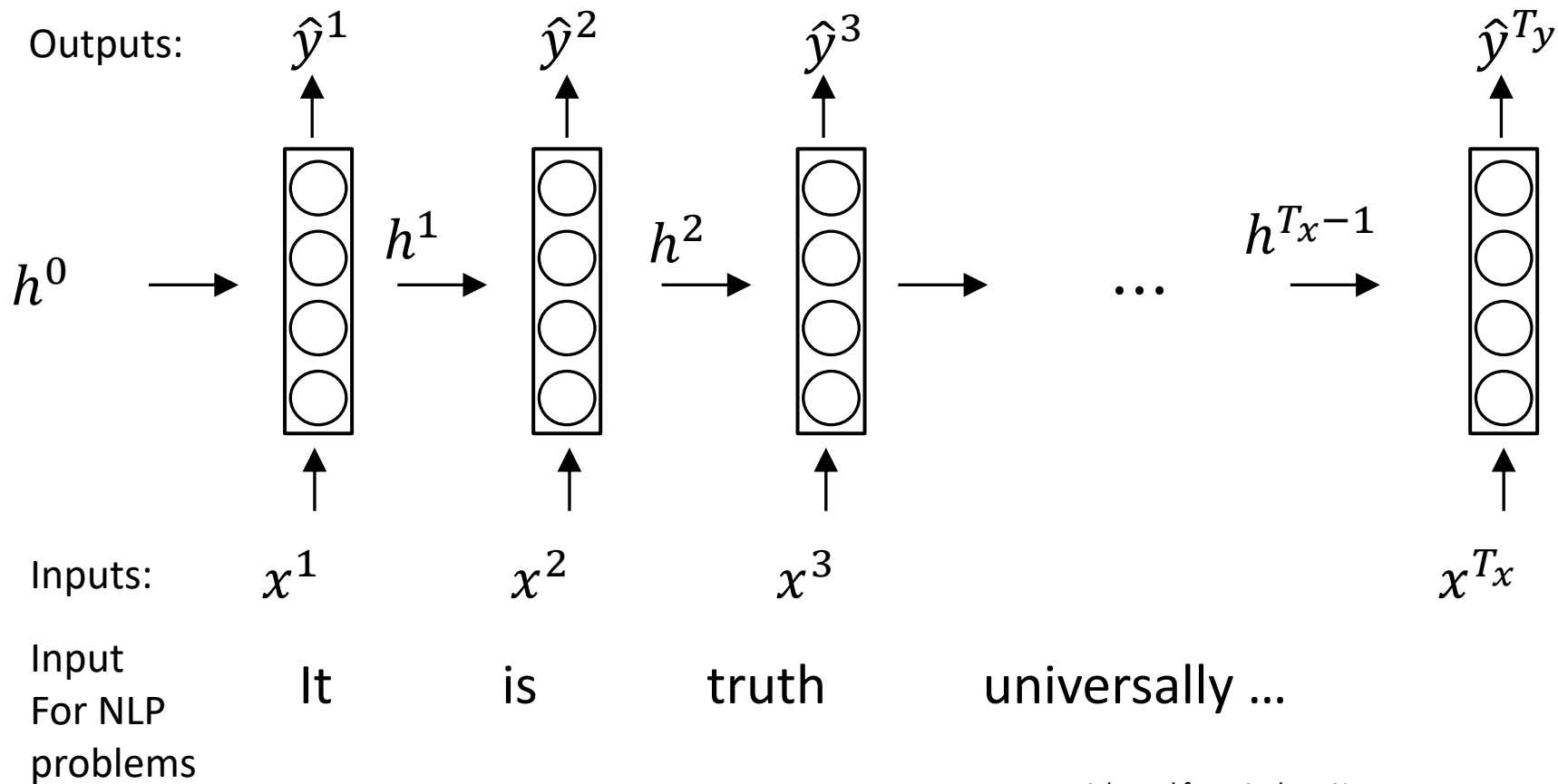
Processing unit is the same for all time moments.
Units has parameters we want to learn!

Forward propagation through Recurrent Neural Network



Processing unit is the same for all time moments.
Units has parameters we want to learn!

Forward propagation through Recurrent Neural Network



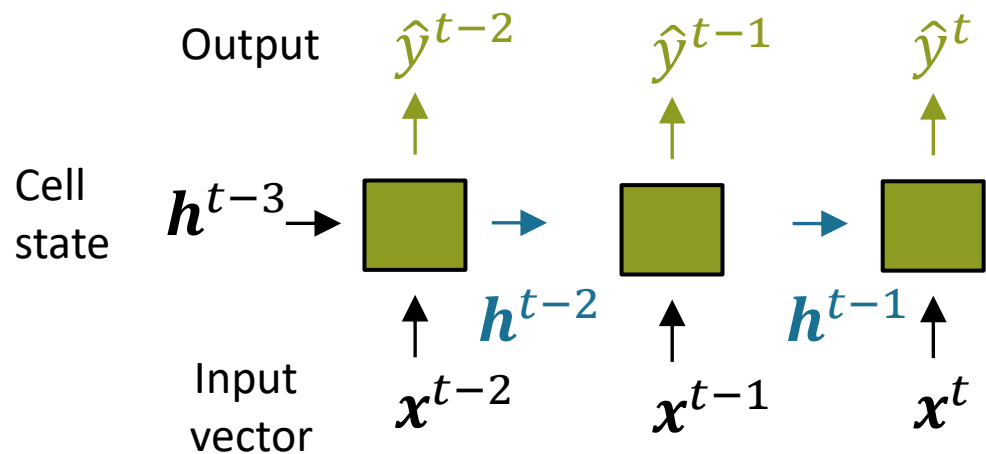
Simple RNN model

At each step:

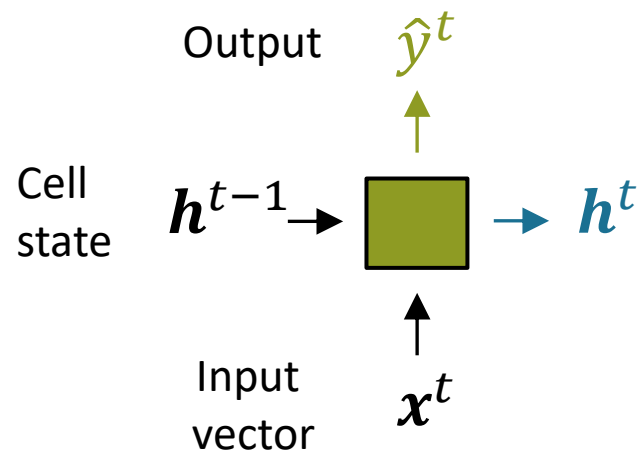
$\hat{y}^t$ - output / model prediction

x^t - input vector / new information

h^t - cell / hidden state



Simple RNN model block



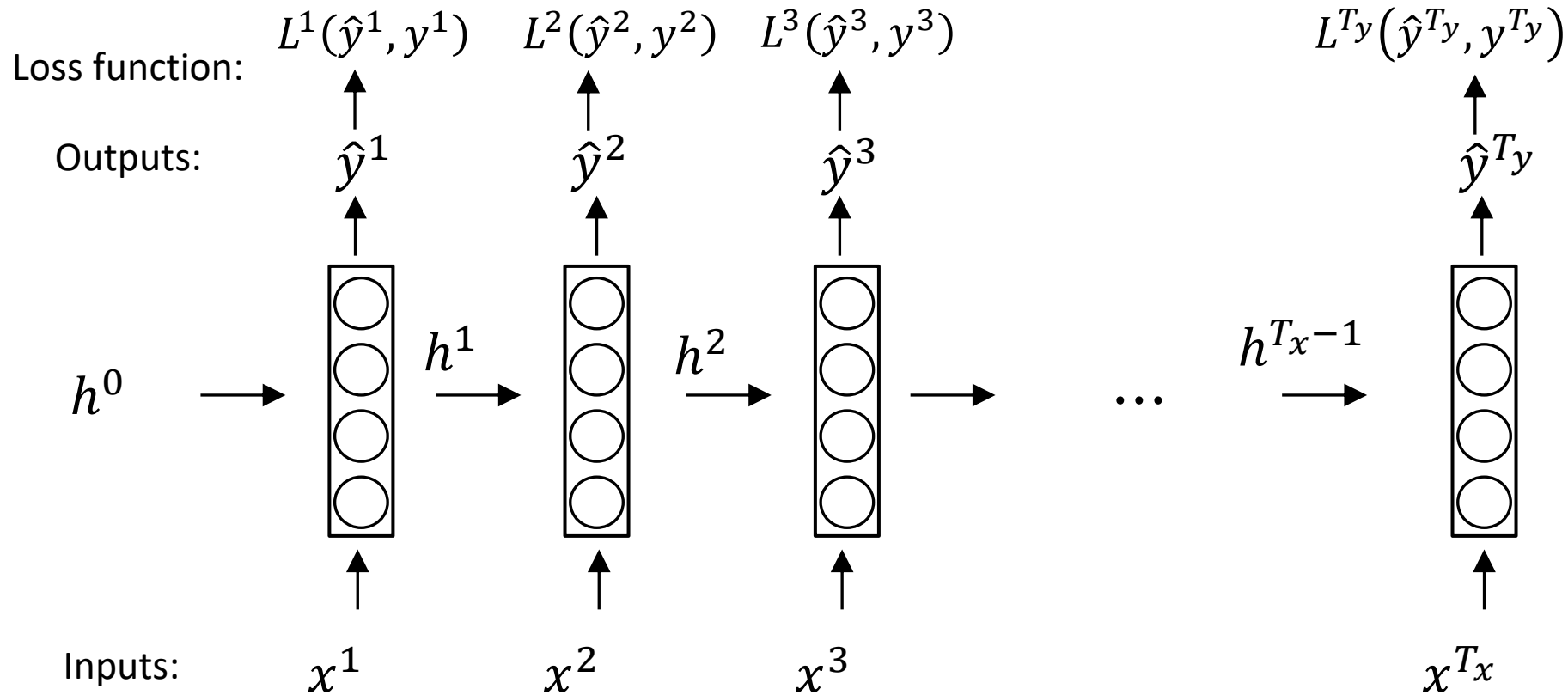
$$\mathbf{h}^t = f_h(\mathbf{x}^t, \mathbf{h}^{t-1})$$

$$\mathbf{h}^t = \tanh(V\mathbf{x}^t + W\mathbf{h}^{t-1} + b_h)$$

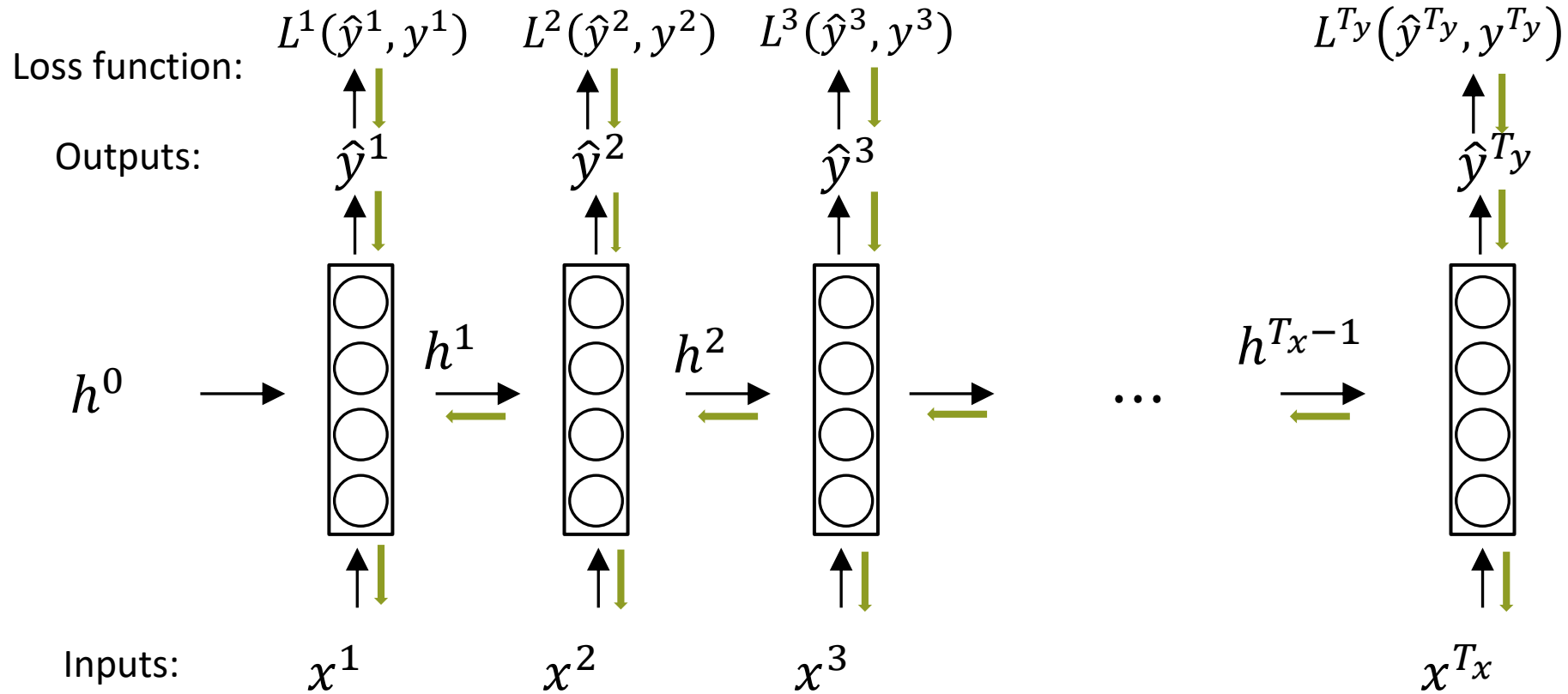
$$\hat{\mathbf{y}}^t = f_y(\mathbf{h}^t)$$

$$\hat{\mathbf{y}}^t = \text{softmax}(U\mathbf{h}^t + b_y)$$

Backward propagation through Recurrent Neural Network



Backward propagation through Recurrent Neural Network

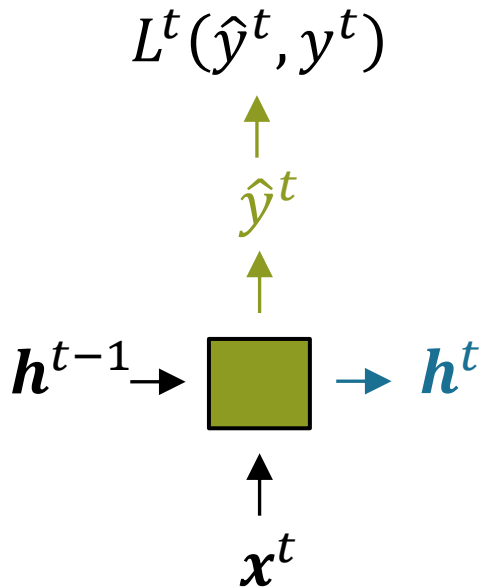


Backpropagation w.r.t. U

$$L = \sum_{i=1}^{T_y} L^i(\hat{y}^i, y^i)$$

$$\frac{\partial L}{\partial U} = \sum_{i=1}^{T_y} \frac{\partial L_i}{\partial U} = \sum_{i=1}^{T_y} \frac{\partial L_i}{\partial \hat{y}^i} \frac{\partial \hat{y}^i}{\partial U}$$

$$\hat{y}^t = \text{softmax}(U\mathbf{h}^t + b_y)$$



Backpropagation w.r.t. W

$$L = \sum_{i=1}^{T_y} L^i(\hat{y}^i, y^i)$$

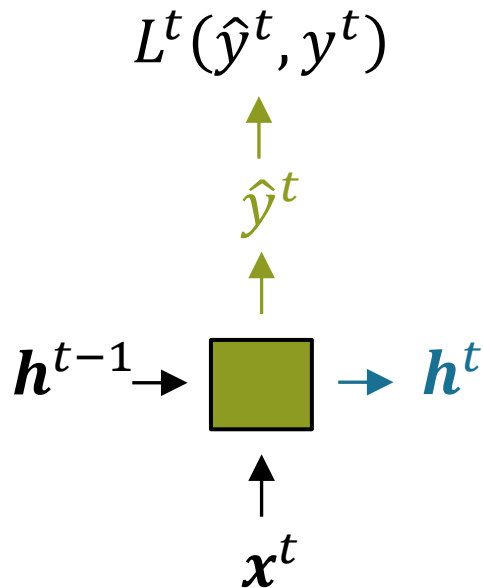
$$\frac{\partial L}{\partial W} = \sum_{i=1}^{T_y} \frac{\partial L_i}{\partial W} = \sum_{i=1}^{T_y} \frac{\partial L_i}{\partial \hat{y}^i} \frac{\partial \hat{y}^i}{\partial W}$$

$$\mathbf{h}^t = \tanh(V\mathbf{x}^t + \mathbf{W}\mathbf{h}^{t-1} + b_h)$$

$$\frac{\partial L_i}{\partial \hat{y}^i} \frac{\partial \hat{y}^i}{\partial W} = \frac{\partial L_i}{\partial \hat{y}^i} \frac{\partial \hat{y}^i}{\partial h_t} \left(\frac{\partial h_t}{\partial W} + \frac{\partial h_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial W} + \right)$$

$$\frac{\partial L_i}{\partial \hat{y}^i} \frac{\partial \hat{y}^i}{\partial W} = \frac{\partial L_i}{\partial \hat{y}^i} \frac{\partial \hat{y}^i}{\partial h_t} \sum_{i=0}^{T_y} \left(\prod_{j=i+1}^{T_y} \frac{\partial h_j}{\partial h_{j-1}} \right) \frac{\partial h_i}{\partial W}$$

$$\hat{y}^t = \text{softmax}(U\mathbf{h}^t + b_y)$$



Problems of classic RNN

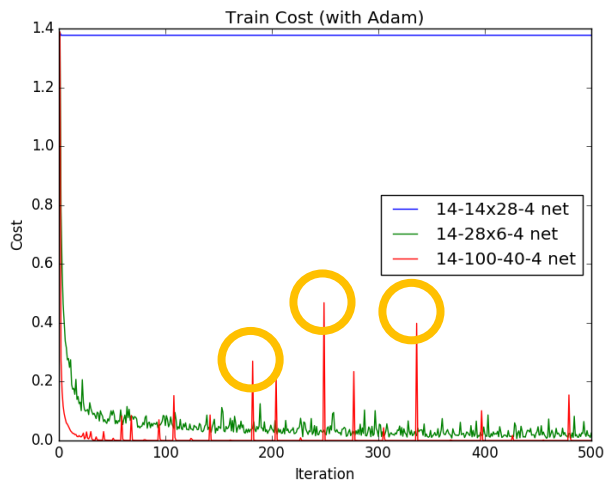
Gradients explosion

Solution:

- Gradient clipping to scale big gradients

$$1. g = \frac{\partial L}{\partial W}$$
$$2. \text{ If } g > t \text{ for some threshold } t:$$
$$g = \frac{t}{\|g\|} g$$

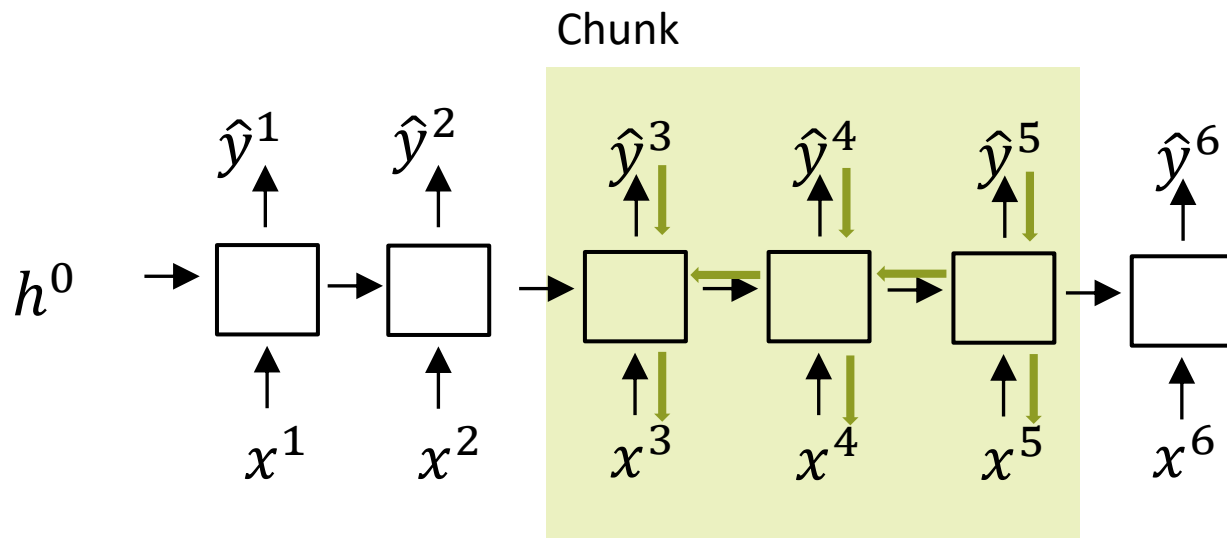
Threshold t is selected given the dynamic of loss function over iterations



Problems of classic RNN

Solution:

- Gradient clipping to scale big gradients
- Truncated backpropagation through time



Problems of classic RNN

Gradients vanishing is a more
serious problem

Many values < 1

Product $\ll 1$

Bias parameters to capture
long-term dependencies

Hard to detect!

Tricks:

Activation functions

- Use ReLU

Parameter initialization

- Initialize weights to identity matrix
- Initialize biases to zero

Another big problem of classic RNN

Problem: Neural networks forget fast, and it is hard to learn long-term dependencies

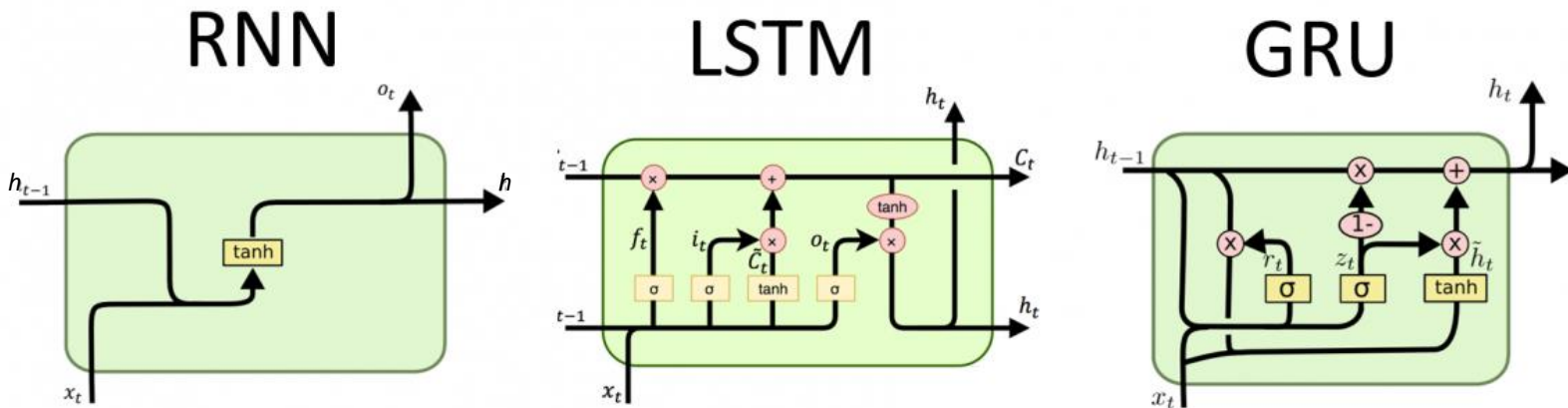
Solution: Gated architectures

More complex recurrent units with gates to control what information is passed through

- GRU (Gated Recurrent Unit)
- LSTM (Long-Short Term Memory)

Better RNN units: LSTM and GRU

- LSTM: long short term memory [1]
- GRU: Gated recurrent unit [2]



1. Schmidhuber, J., & Hochreiter, S. (1997). Long short-term memory. *Neural Comput*, 9(8), 1735-1780.
2. Cho, K., Van Merriënboer et al. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*.

Attention mechanism

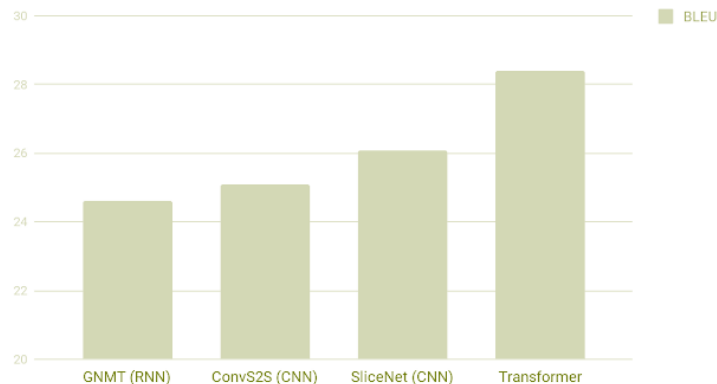
New state of the art: **attention** is all we need

*all you
need is
love.*

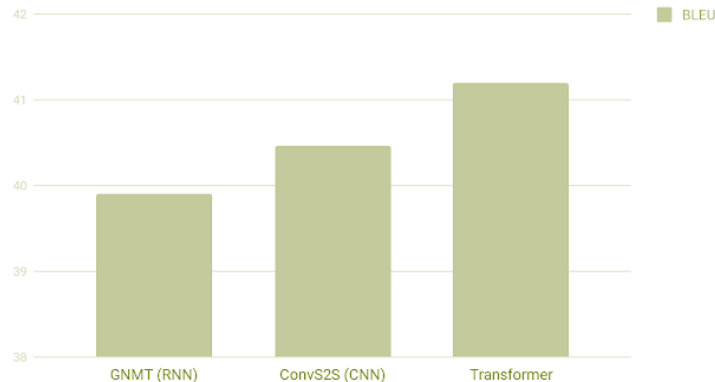
attention

Higher BLEU scores are better

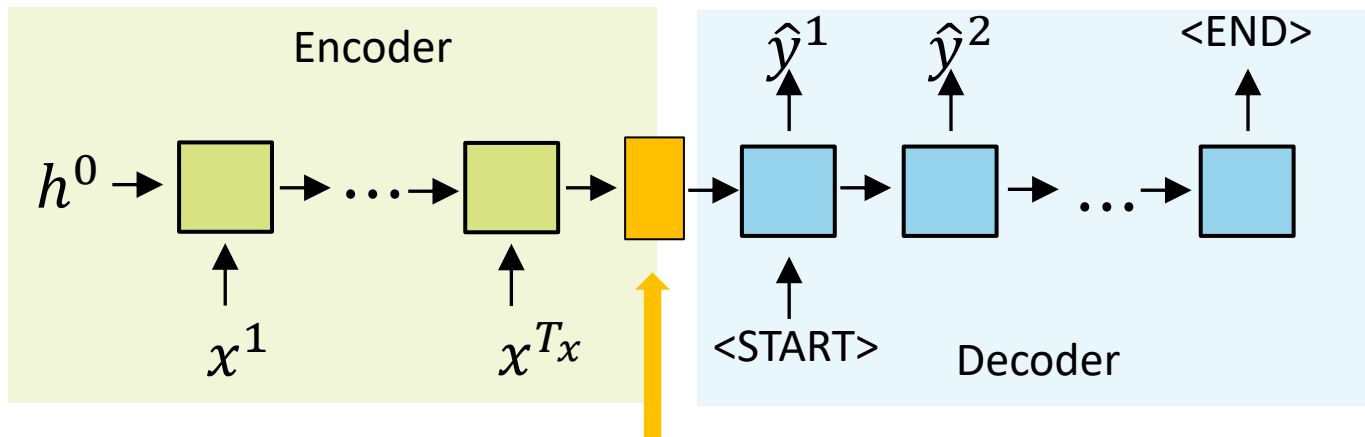
English German Translation quality



English French Translation Quality



Bottleneck in seq2seq models



All information about the sequence is in this vector

Attention

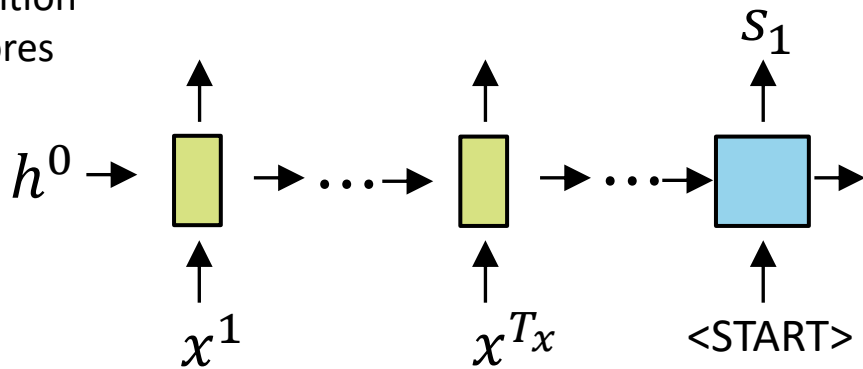
- Solution to the bottleneck problem
- Direction connection between parts of input and output sequence

Sequence 2 sequence with attention

Attention
output

Attention
distribution

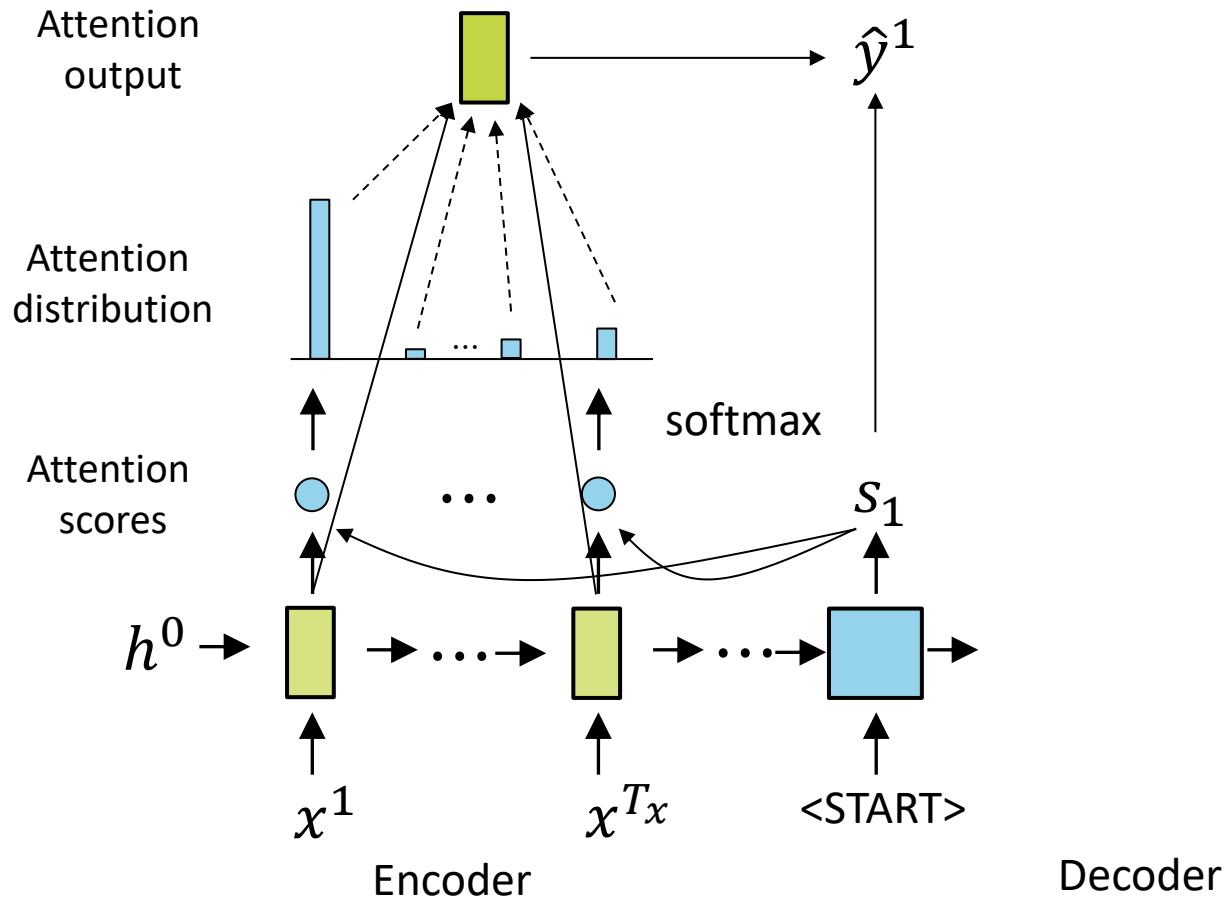
Attention
scores



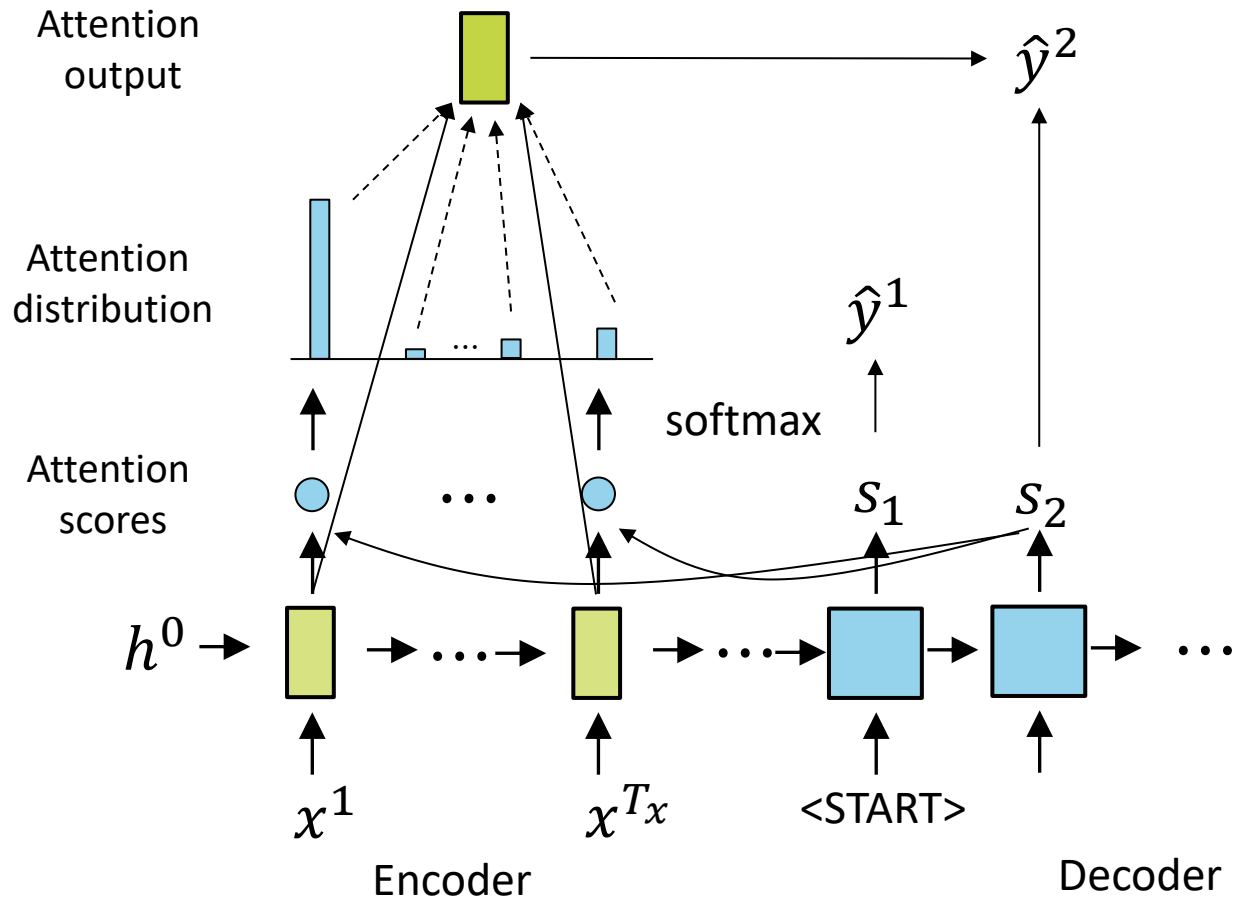
Encoder

Decoder

Sequence 2 sequence with attention



Sequence 2 sequence with attention



Attention: formulas

- First RNN produces encoder hidden states $\mathbf{h}_1, \dots, \mathbf{h}_{T_x} \in \mathbb{R}^h$
- Decoder hidden state $\mathbf{s}_t \in \mathbb{R}^h$ at time step t
- Attention scores for step t :
 $\mathbf{e}^t = [\mathbf{s}_t^T \mathbf{h}_1, \dots, \mathbf{s}_t^T \mathbf{h}_{T_x}] \in \mathbb{R}^{T_x}$
- Softmax to get attention distribution: all values are positive, sum of all values is 1:

$$\boldsymbol{\alpha}^t = \text{softmax}(\mathbf{e}^t) \in \mathbb{R}^{T_x}$$

- Attention output $\mathbf{a}_t$ is a weighted sum of hidden states:

$$\mathbf{a}_t = \sum_{i=1}^{T_x} \alpha_i^t \mathbf{h}_i \in \mathbb{R}^h$$

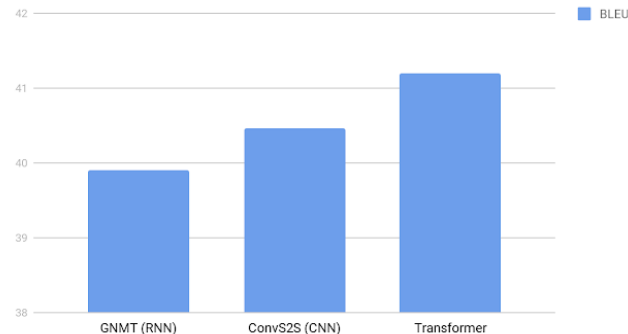
- We concatenate the attention output $\mathbf{a}_t$ with the decoder hidden state $\mathbf{s}_t$ and proceed to the non-attention part of our seq2seq model

$$[\mathbf{a}_t, \mathbf{s}_t] \in \mathbb{R}^{2h}$$

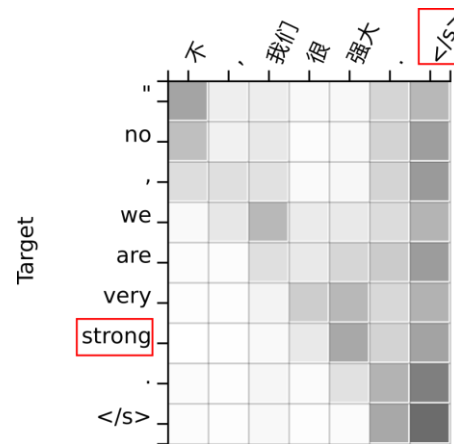
Attention is just great

- Significantly improves performance of NMT
- Solves the bottleneck problem
 - All encoder tokens are connected to all decoder tokens
- No more vanishing gradients
 - All to All connection
- Provides some interpretability
 - see alignment figure
- Similar to RNN seq2seq, but greater!

English French Translation Quality



BLEU scores (higher is better) of single models on the standard WMT newstest2014 English to French translation benchmark.



Attention is a general deep learning idea

We can use attention in many architectures and many tasks

- Other NLP problems
- Sequential data processing
- Graph Neural Networks

Key value interpretation:

s_i - query to a database

Hidden state of the decoder

k_i - keys in the database

Hidden state of the encoder

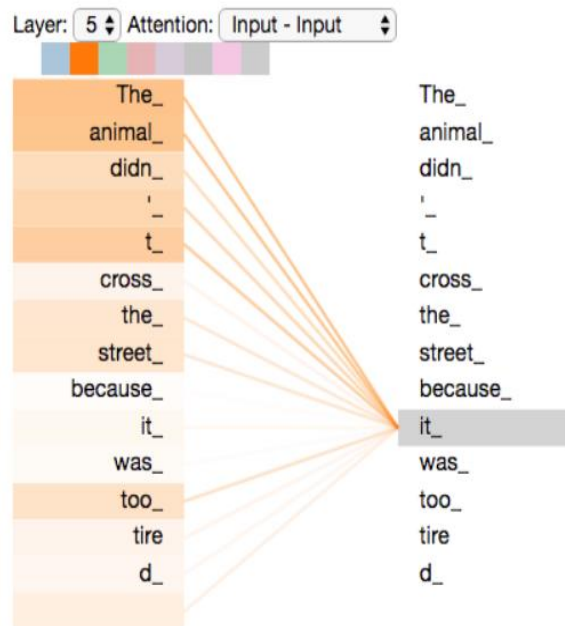
h_i - values in the database

Hidden state of the encoder

- Calculate correspondence $e(s_i, k_i)$
- Calculate weights on the base of correspondence values
- Extract information as weighted sum of values $\sum_{i=1}^n \alpha_i \mathbf{h}_i$

Interpretability of attention

Remove a token i^* with max attention...



Remove i^* : Decision flip?

Remove random: Decision flip?

		Yahoo		IMDB	
		Yes	No	Yes	No
Remove i^* : Decision flip?	Yes	0.5	8.7	2.2	12.2
	No	1.3	89.6	1.4	84.2

		Amazon		Yelp	
		Yes	No	Yes	No
Remove i^* : Decision flip?	Yes	2.7	7.6	1.5	8.9
	No	2.7	87.1	1.9	87.7

Figure source:
<https://jalammar.github.io/illustrated-transformer/>,
<https://github.com/jessevig/bertviz>

Serrano, Sofia, and Noah A. Smith. "Is attention interpretable?." arXiv preprint arXiv:1906.03731 (2019). ACL 2019.

General attention idea

Attention

We need a representation of an object.

A “coarse” description of an object is available.

For other objects we have more detailed stored information.

We use attention to extract these information

Single-object key value interpretation

q_i - query to a database

k_j - keys in the database

v_j - values in the database

We calculate attention scores

$$\alpha_j = e(q_i, k_j) = s_i^T k_j \text{ (other distances also possible)}$$

$$\alpha = \text{softmax}(\alpha)$$

Then we extract the information as weighted sum of values

$$a_i = \sum_{j=1}^{T_x} \alpha_j v_j$$

Matrix key value interpretation

q_i - query to a database

k_j - keys in the database

v_j - values in the database

We calculate correspondences

$$A(q, K, V) = \sum_j \frac{\exp(q_i^T k_j)}{\sum_l \exp(q_i^T k_l)} v_j$$

$$A(Q, K, V) = \text{softmax}(QK^T)V$$

“Databases” for the attention

- Nodes in a graph
- Tokens in a sequence (we need to specify the position, as the order is important)
- Sets of objects

Single-object key value interpretation

q_i - query to a database

Hidden state of the *decoder*

k_j - keys in the database

Hidden state of the *encoder*

v_j - values in the database

Hidden state of the *encoder*

We calculate attention scores

$$\alpha_j = e(q_i, k_j) = s_i^T k_j \text{ (other distances also possible)}$$

$$\alpha = \text{softmax}(\alpha)$$

Then we extract the information as weighted sum of values

$$a_i = \sum_{j=1}^{T_x} \alpha_j v_j$$

Scaled attention values

For large dimension of the space of keys d_k :

- Large variances dot products $\mathbf{q}_i^T \mathbf{k}_j$
- Softmax only pays attention to some keys
- Gradients are small, hard to learn

Old formula:

$$A(Q, K, V) = \text{softmax}(QK^T)V$$

New scaled formula:

$$A(Q, K, V) = \text{softmax}(QK^T/\sqrt{d_k})V$$



Transformers with self-attention

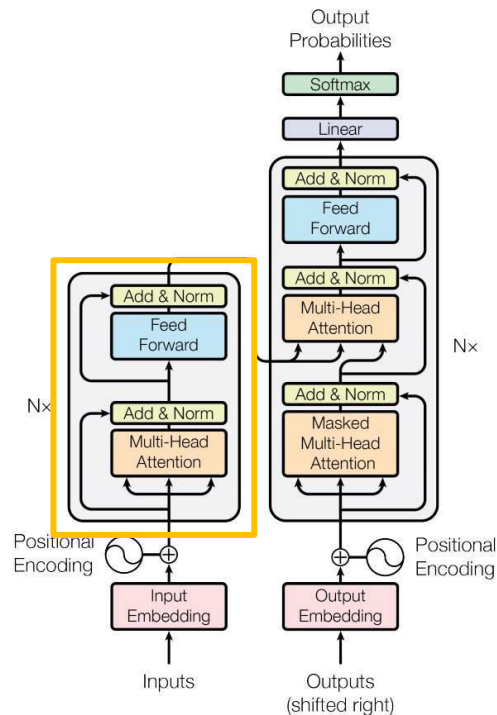
Transformer is based on the same idea

Now we completely drop RNN part

Also we repeat *self-attention* many times

Further we'll consider separate parts:

- Multi-head attention
- Feed Forward



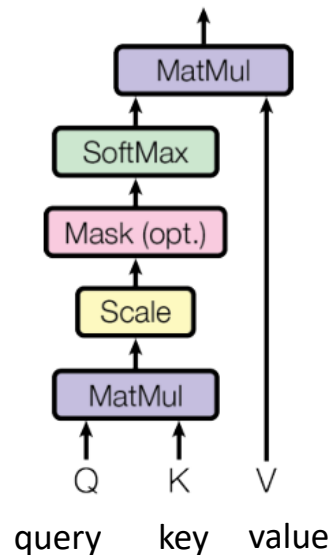
Attention / Self-attention block

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

d_k is the dimension of query and key,
we scale to take control of large values of dot-product in high dimensions

A possible option is to replace scaled dot-product used here with additive attention: a single-hidden layer neural network.

Scaled Dot-Product Attention



Self-attention block

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

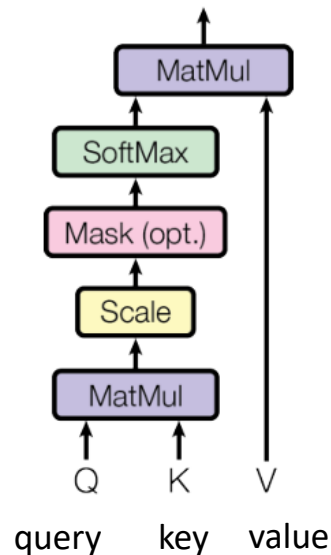
We produce queries, keys, and values using initial word embeddings

$$Q = XW^Q, \dim(W^Q) = d_x \times d_q,$$

$$K = XW^K, \dim(W^K) = d_x \times d_k,$$

$$V = XW^V, \dim(W^V) = d_x \times d_v,$$

Scaled Dot-Product Attention



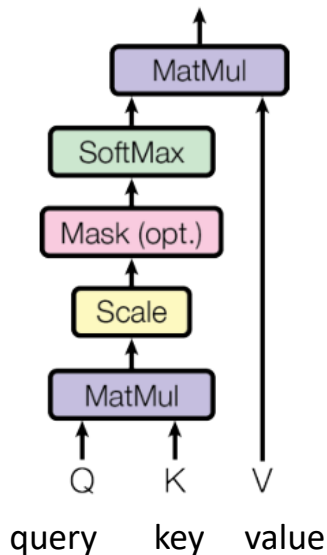
Multi-Head attention

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

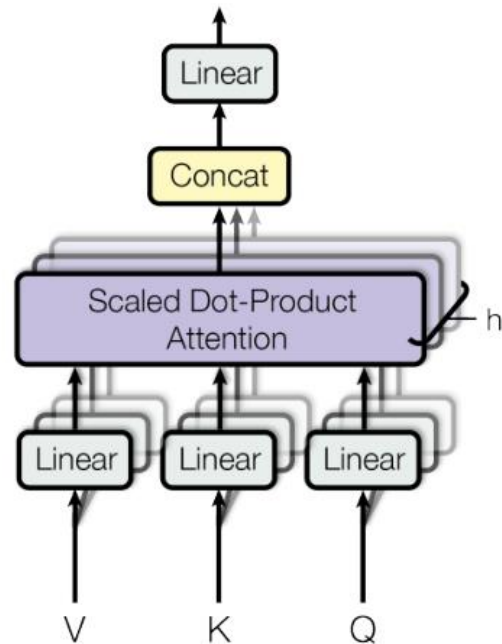
$$\text{where head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

h heads in total

Scaled Dot-Product Attention



Multi-Head Attention



“Attention is all you need” paper

Full block

Two linear transformation with ReLU activation in between

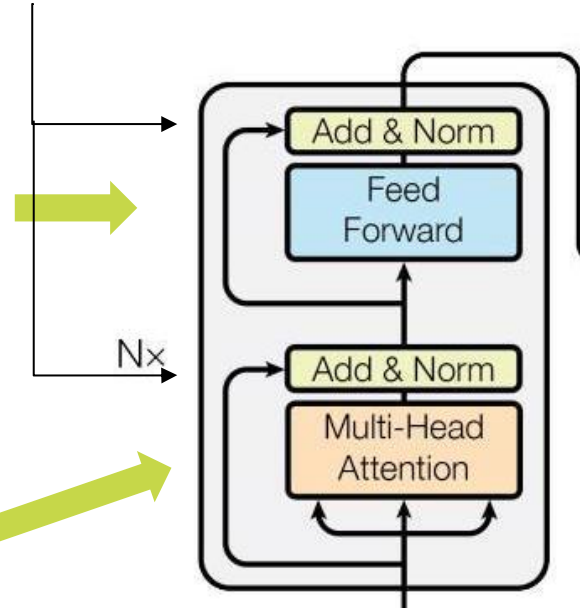
$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Multi-head attention

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

$$\text{LayerNorm}(x + \text{Sublayer}(x))$$



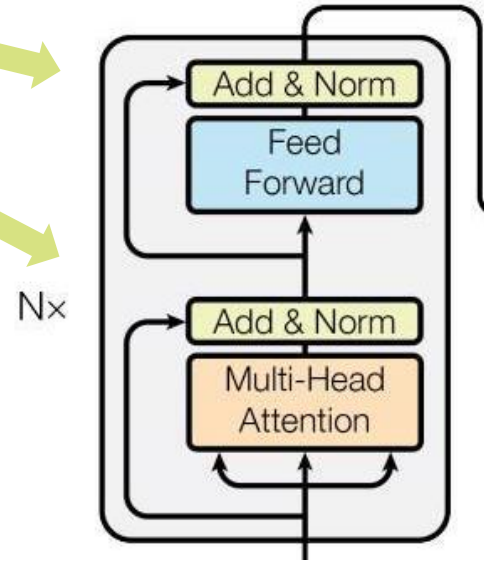
There are 6 consecutive *Full blocks* in the paper transformer architecture

Full block: normalization and residual connection

$$\text{LayerNorm}(x + \text{Sublayer}(x))$$

Why? Speeds up training!

- Similar to Batch Normalization
- But can be used with batch size 1
- Can be used with RNNs and Transformers



$$\mu^l = \frac{1}{H} \sum_{i=1}^H a_i^l \quad \sigma^l = \sqrt{\frac{1}{H} \sum_{i=1}^H (a_i^l - \mu^l)^2}$$

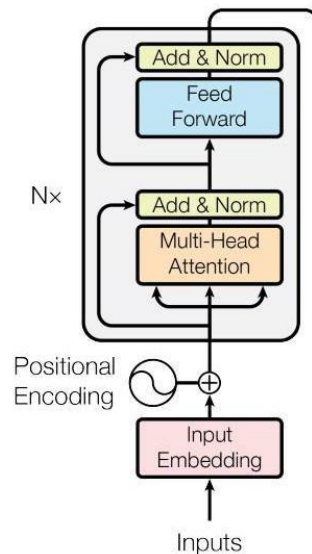
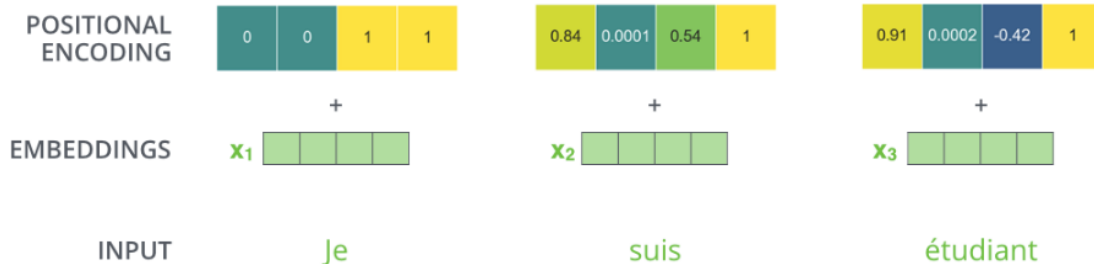
Position encoding

In addition to usual embeddings of inputs we use position encoding to capture position

They are not one-hot vectors, as we want to handle various-length sequences

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$



Real example of positional encoding with a toy embedding size of 4

Transformer training

Self-supervised learning: We don't need labeled examples, we just create them

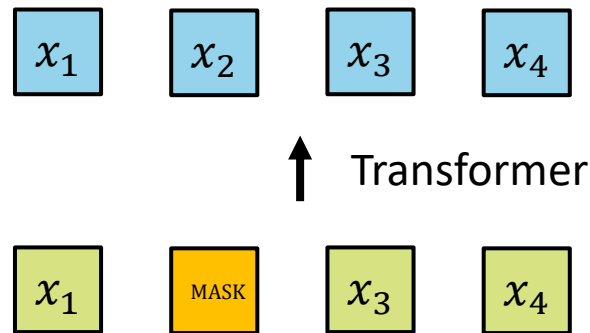
- Masked language model:

Replace random tokens with masks, try to reconstruct them using a Neural network

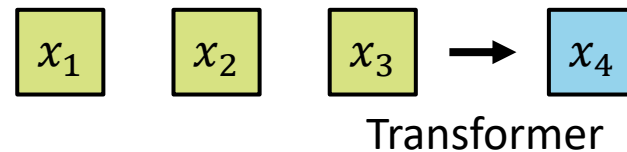
- Next token prediction

Predict next token

BERT



GPT



Efficiency of transformers

Brown, T. B., Mann, B., Ryder, N. et al. (2020). Language models are few-shot learners. *arXiv preprint arXiv:2005.14165*.

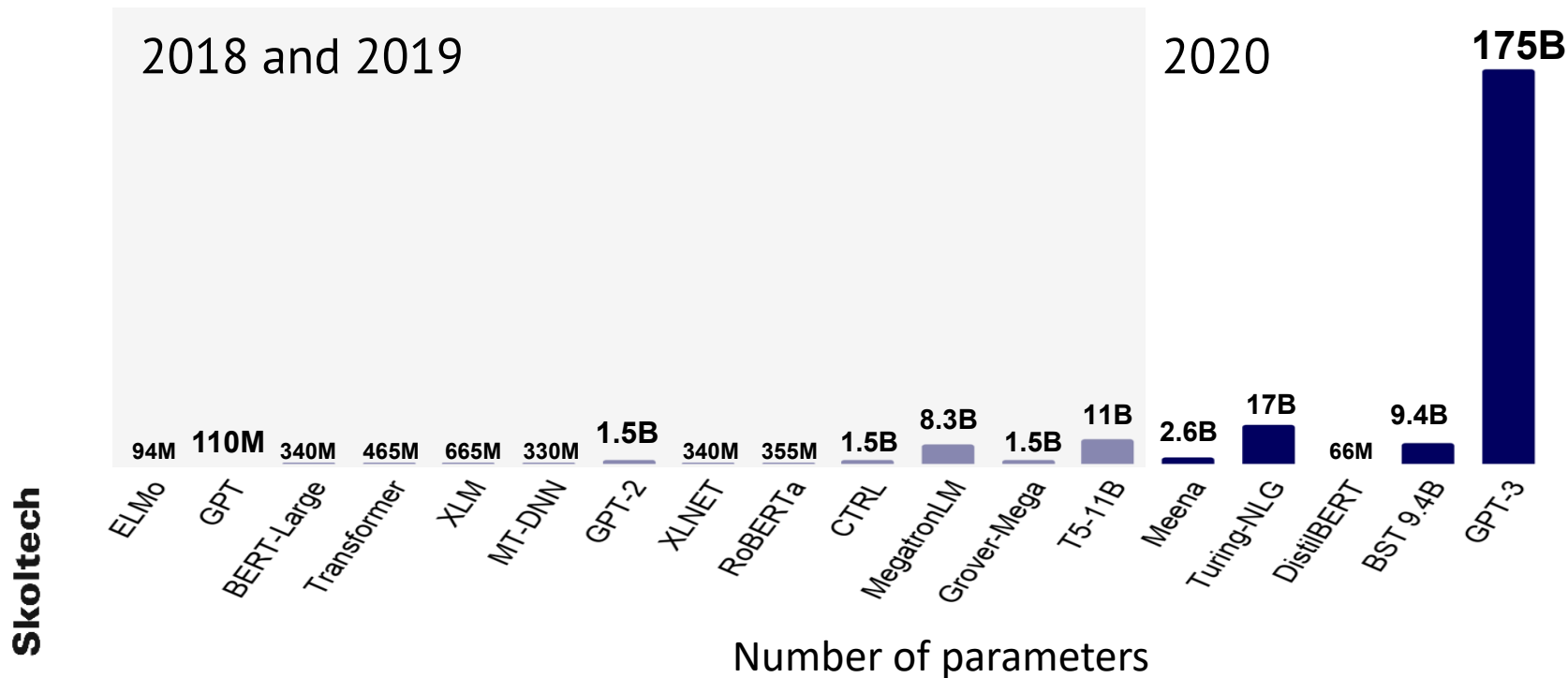
Model Name	n_{params}	n_{layers}	d_{model}	n_{heads}	d_{head}	Batch Size	Learning Rate
GPT-3 Small	125M	12	768	12	64	0.5M	6.0×10^{-4}
GPT-3 Medium	350M	24	1024	16	64	0.5M	3.0×10^{-4}
GPT-3 Large	760M	24	1536	16	96	0.5M	2.5×10^{-4}
GPT-3 XL	1.3B	24	2048	24	128	1M	2.0×10^{-4}
GPT-3 2.7B	2.7B	32	2560	32	80	1M	1.6×10^{-4}
GPT-3 6.7B	6.7B	32	4096	32	128	2M	1.2×10^{-4}
GPT-3 13B	13.0B	40	5140	40	128	2M	1.0×10^{-4}
GPT-3 175B or “GPT-3”	175.0B	96	12288	96	128	3.2M	0.6×10^{-4}

Table 2.1: Sizes, architectures, and learning hyper-parameters (batch size in tokens and learning rate) of the model: which we trained. All models were trained for a total of 300 billion tokens.

	Mean accuracy	95% Confidence Interval (low, hi)	t compared to control (p -value)	“I don’t know” assignments
Control (deliberately bad model)	86%	83%–90%	-	3.6 %
GPT-3 Small	76%	72%–80%	3.9 ($2e-4$)	4.9%
GPT-3 Medium	61%	58%–65%	10.3 ($7e-21$)	6.0%
GPT-3 Large	68%	64%–72%	7.3 ($3e-11$)	8.7%
GPT-3 XL	62%	59%–65%	10.7 ($1e-19$)	7.5%
GPT-3 2.7B	62%	58%–65%	10.4 ($5e-19$)	7.1%
GPT-3 6.7B	60%	56%–63%	11.2 ($3e-21$)	6.2%
GPT-3 13B	55%	52%–58%	15.3 ($1e-32$)	7.1%
GPT-3 175B	52%	49%–54%	16.9 ($1e-34$)	7.8%

Table 3.11: Human accuracy in identifying whether short (~200 word) news articles are model generated. We find that human accuracy (measured by the ratio of correct assignments to non-neutral assignments) ranges from 86% on the control model to 52% on GPT-3 175B. This table compares mean accuracy between five different models, and shows the results of a two-sample T-Test for the difference in mean accuracy between each model and the control model (an unconditional GPT-3 Small model with increased output randomness).

Transformer models are monstrous



Training cost estimation: 10-50 MLN US\$ for GPT-3

Self-attention has high complexity

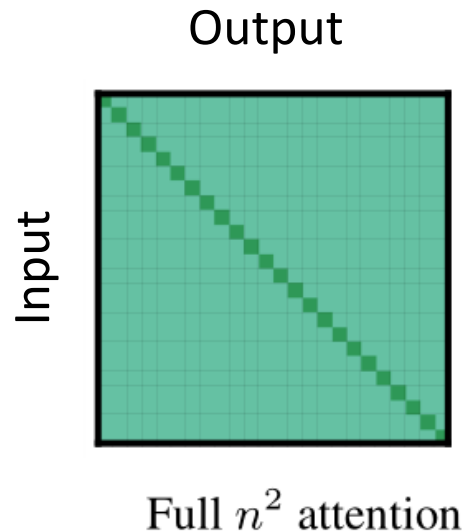
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Memory complexity is $O(d_x^2)$

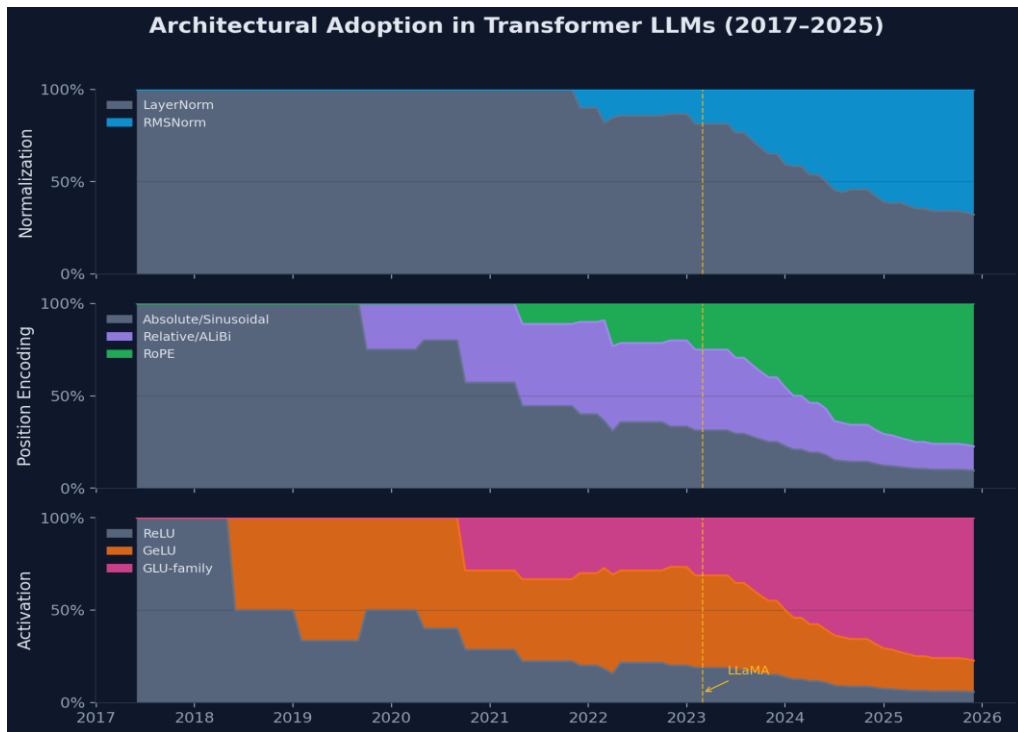
Computational complexity is $O(d_x^2)$

Max sequence size in popular models (e.g. BERT) is only $n = d_x = 512$ tokens

In modern models, tokens are parts of words



Crystallization of transformer architecture: 2017-2025: normalization layer



RMSNorm

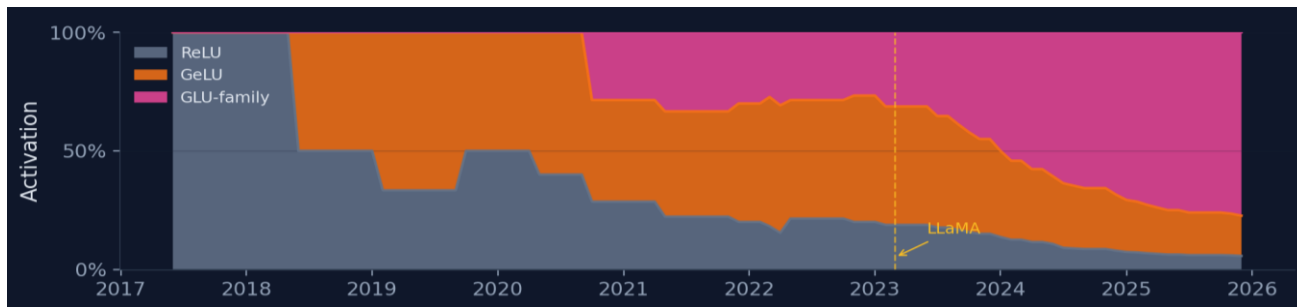
$$\text{LayerNorm}(x) = \frac{x - \mu}{\sigma} \odot \gamma + \beta$$

$$\text{RMSNorm}(x) = \frac{x}{\text{RMS}(x)} \odot \gamma, \quad \text{RMS}(x) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2}$$

What: no mean scaling for features

Why: 10-15% computational saving without performance drop

Crystallization of transformer architecture: 2017-2025 : fully connected/normalization layer



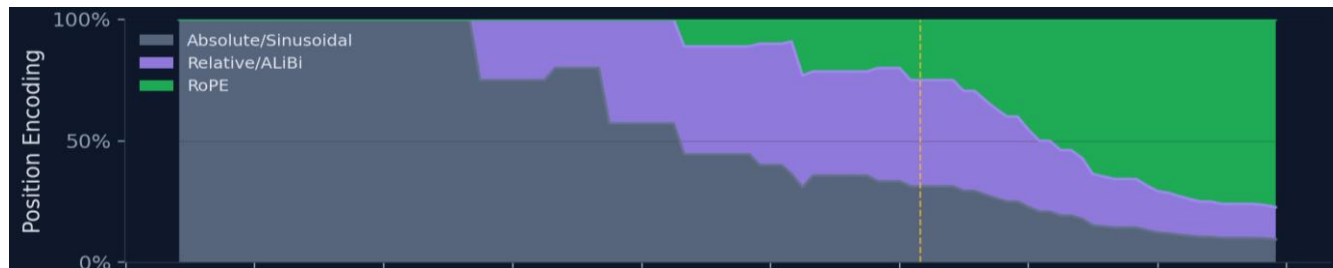
$$\text{FFN}(x) = \text{GeLU}(xW_1)W_2$$

$$\text{SwiGLU}(x, W_1, W_3, W_2) = (\text{SiLU}(xW_1) \odot xW_3)W_2$$

What: Gating with SiLU activation

Why: Learn to amplify or suppress improving expressivity

Crystallization of transformer architecture: 2017-2025 : rotary positional encoding



$$q_n = W_q(x_n + p_n), \quad k_m = W_k(x_m + p_m)$$

$$q_n^T k_m = (W_q x_n)^T (W_k x_m) + (W_q x_n)^T (W_k p_m) + (W_q p_n)^T (W_k x_m) + (W_q p_n)^T (W_k p_m)$$

$$R_n = \begin{pmatrix} \cos n\theta & -\sin n\theta \\ \sin n\theta & \cos n\theta \end{pmatrix}$$

$$q_n = R_n W_q x_n, \quad k_m = R_m W_k x_m$$

Why: Better intuition behind

Take-home messages

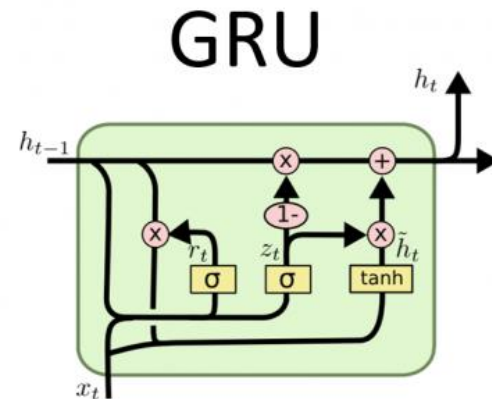
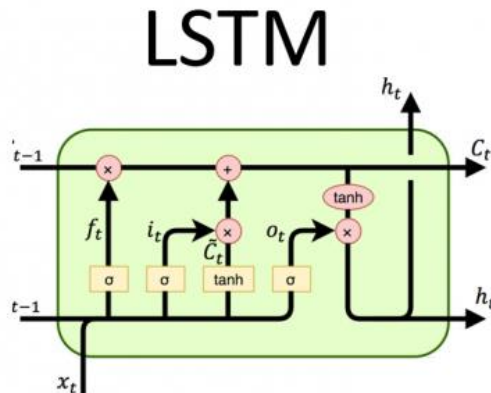
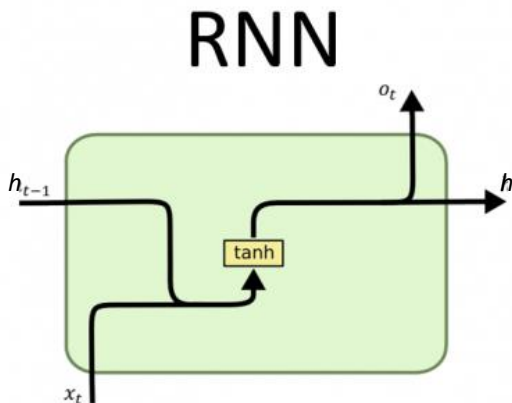
- Attention mechanism is for “information extraction”
- Transformer architecture is a multi-layer architecture based on the self attention layer
- We can adopt attention idea to other modalities like graphs or images

Selection of RNN architecture



Better RNN units: LSTM and GRU

- LSTM: long short term memory [1]
- GRU: Gated recurrent unit [2]



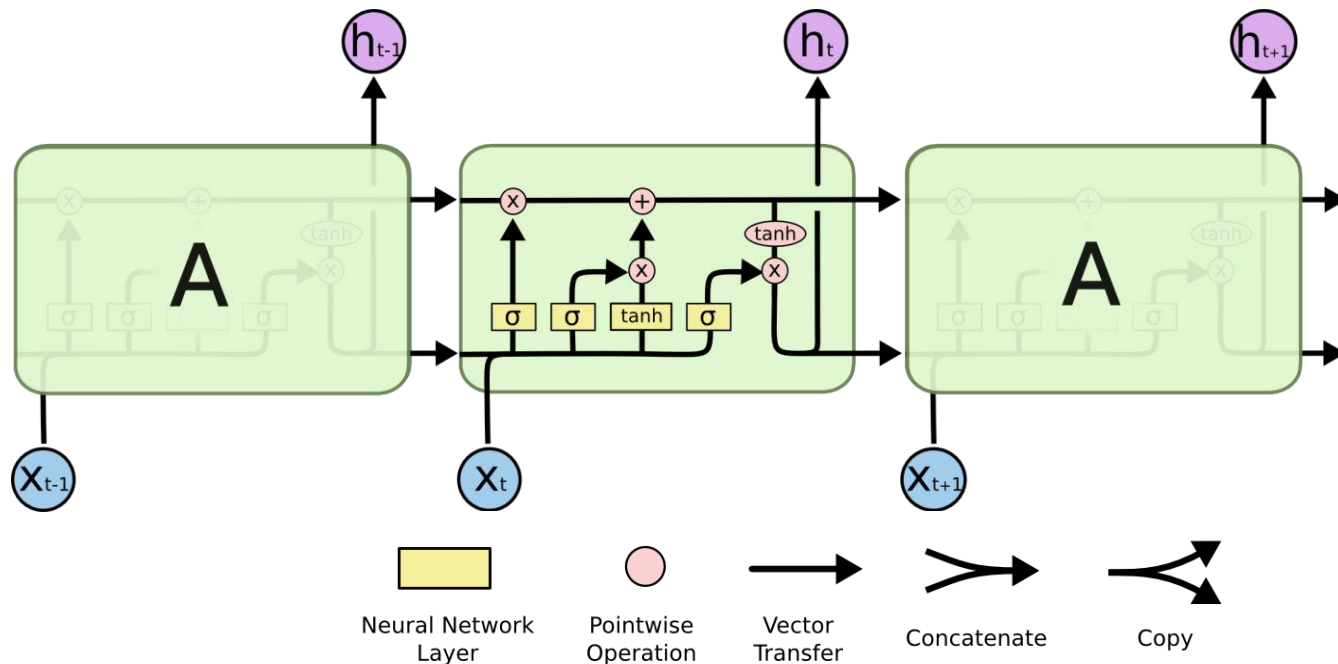
1. Schmidhuber, J., & Hochreiter, S. (1997). Long short-term memory. *Neural Comput*, 9(8), 1735-1780.
2. Cho, K., Van Merriënboer et al. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*.

Details on how LSTM works

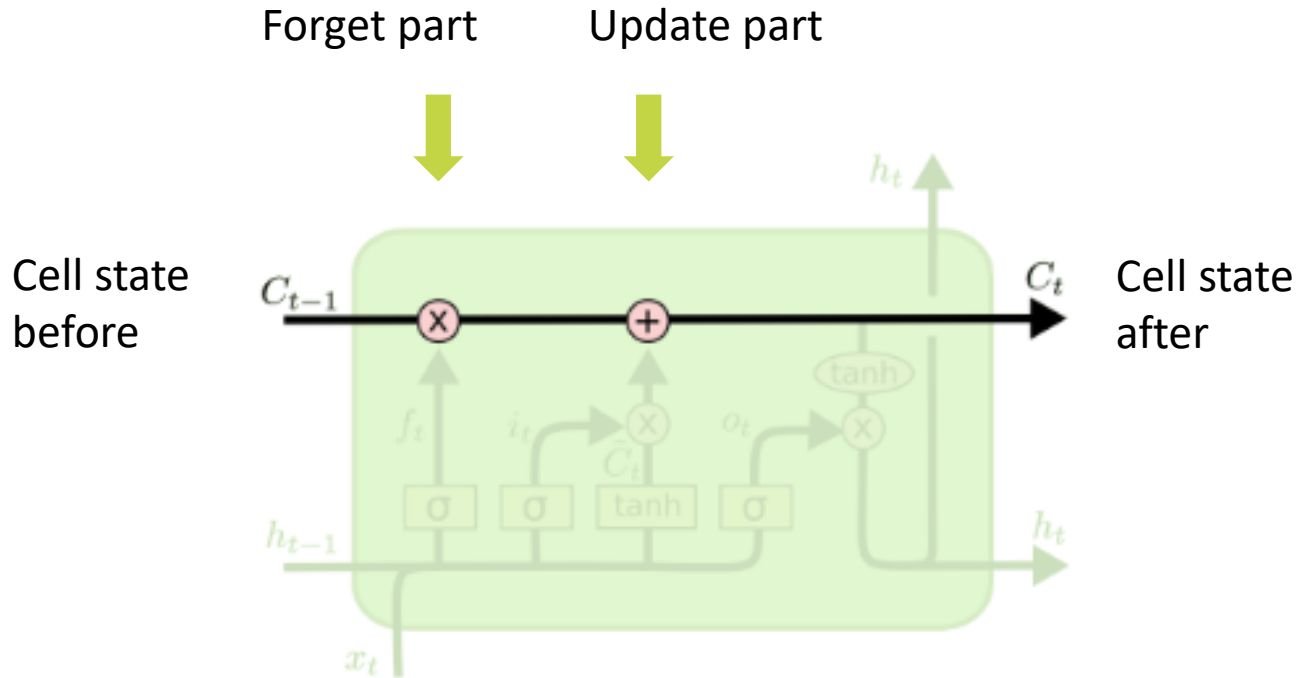
Remembering information for long periods of time is practically the default behavior of LSTM



LSTM was proposed by J. Schmidhuber group in 1991

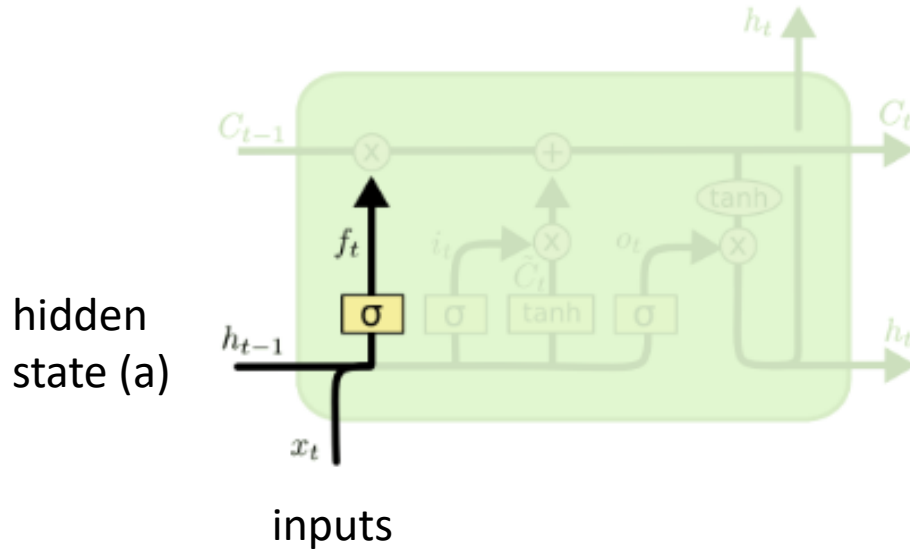


Long term memory part – Cell state



Forget part

Identify how much should we forget: sigmoid returns value between 0 and 1

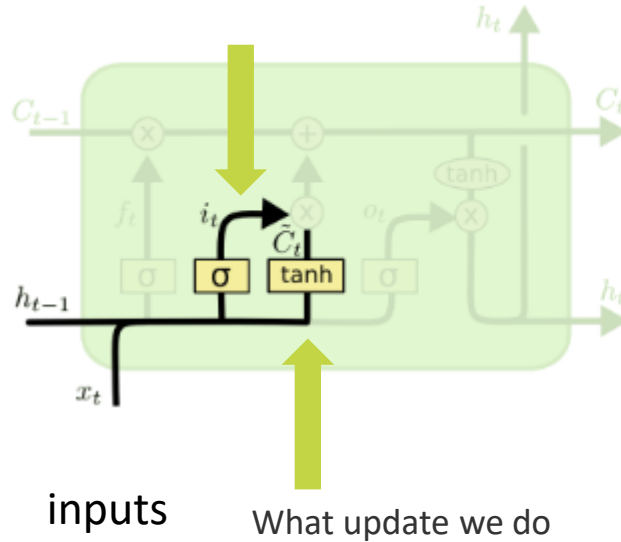


$$f_t = \sigma (W_f \cdot [h_{t-1}, x_t] + b_f)$$

Update part

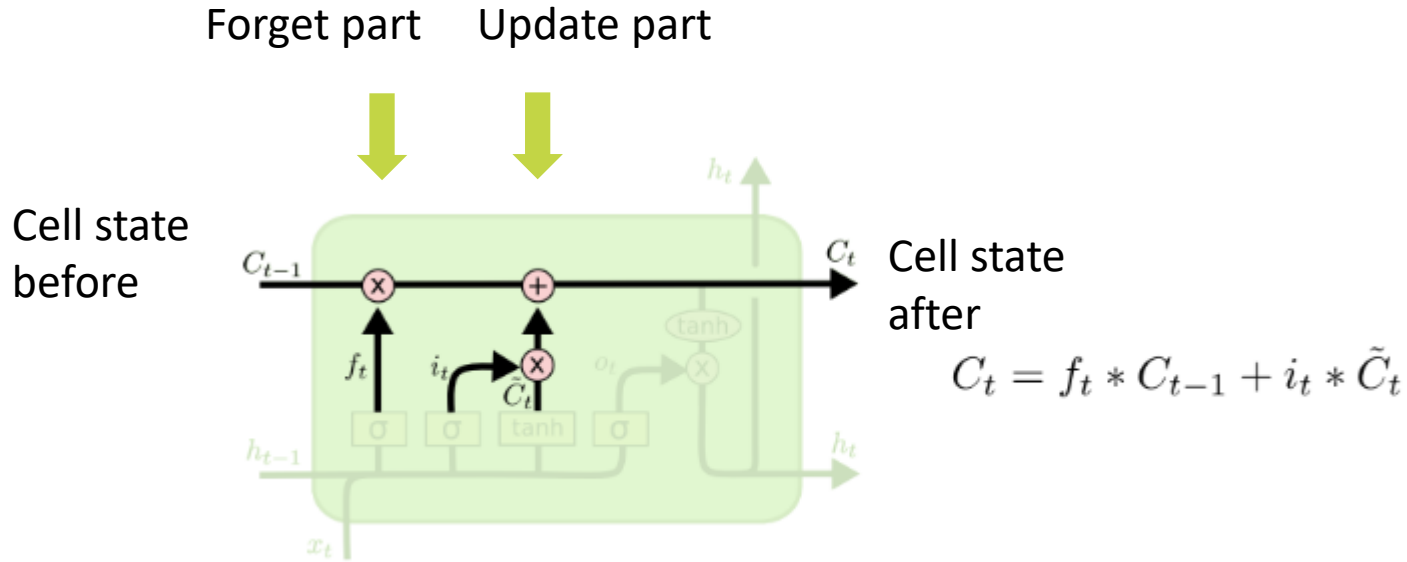
Identify how much should we update:
sigmoid returns value between 0 and 1

hidden
state (a)

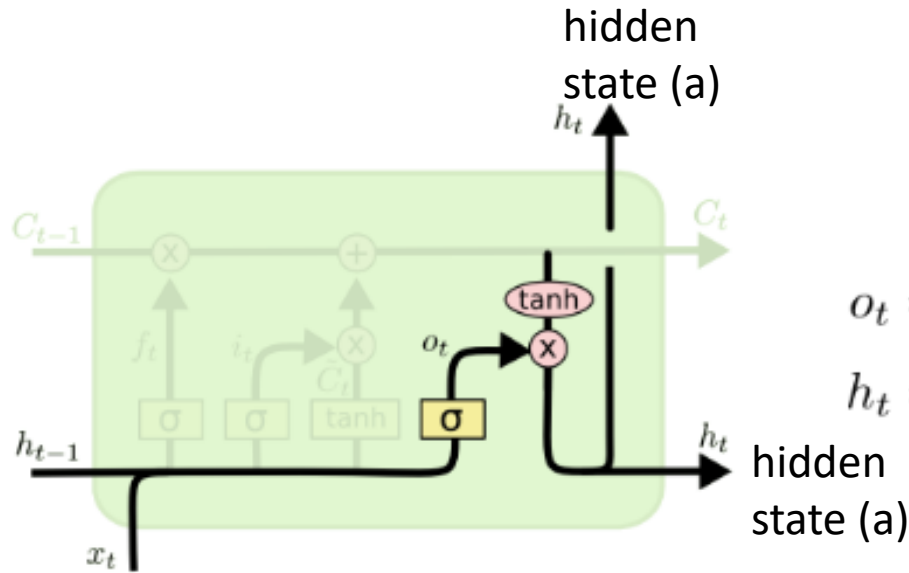


$$i_t = \sigma (W_i \cdot [h_{t-1}, x_t] + b_i)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

Long term memory part – Cell state



Update everything else

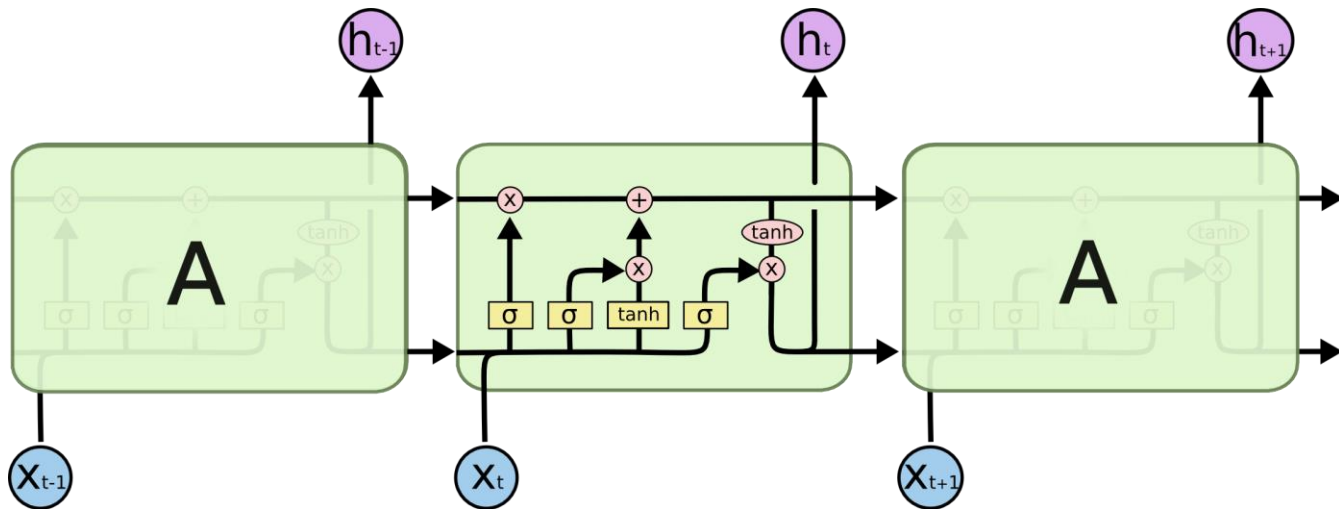


$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

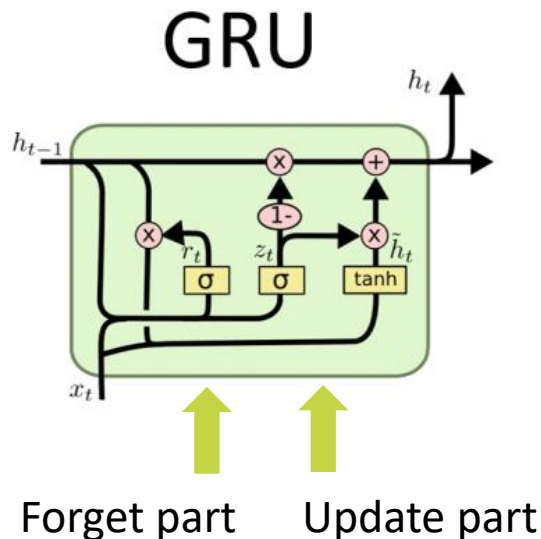
Details on how LSTM works

- There are cell and hidden (activation) states
- LSTM block forgets and updates cell state during processing at one block

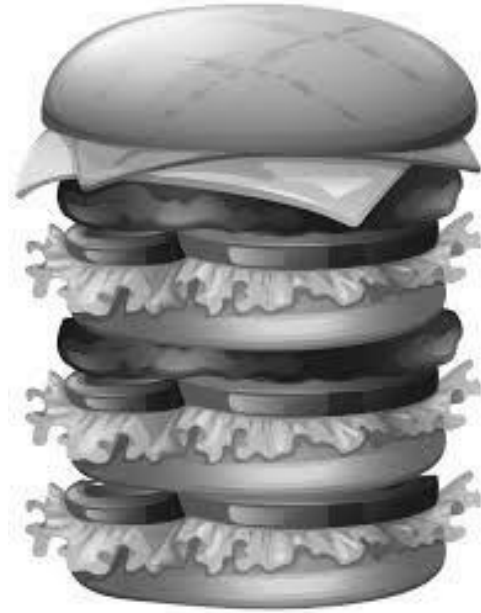


GRU – Gated Recurrent Unit

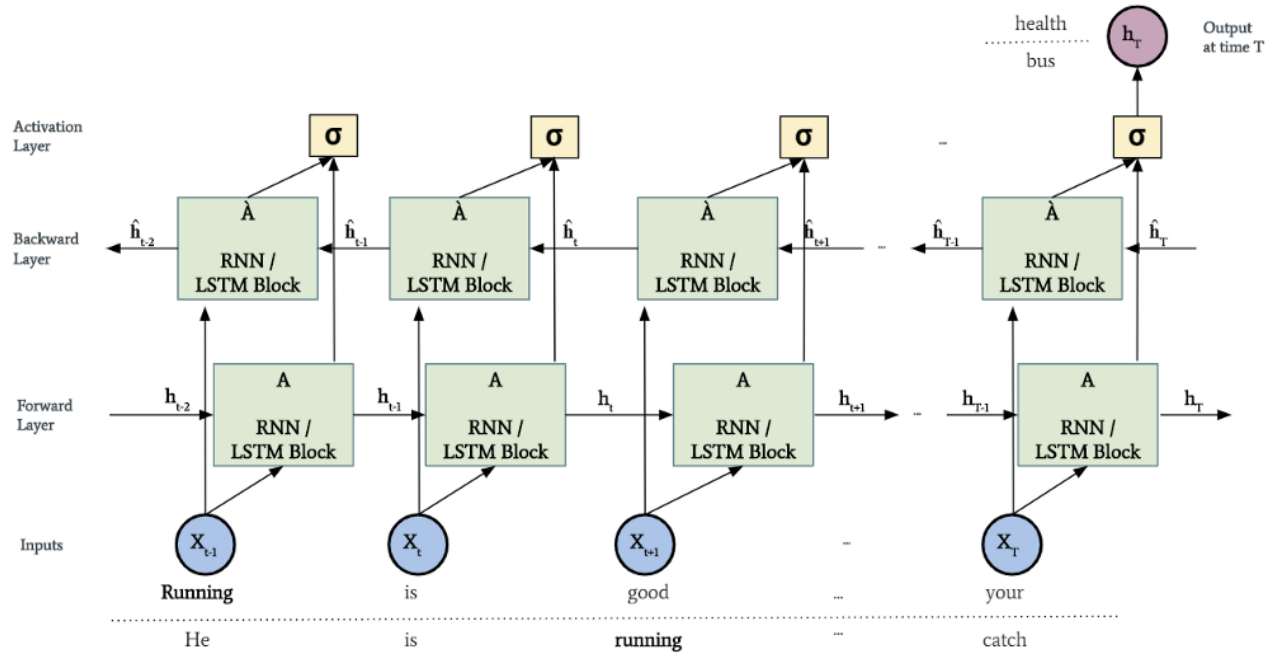
- Update gate – what to pay attention to
- Reset gate – what to forget
- Slightly worse than LSTM for NLP – but not in all problems
- Simpler and cheaper than LSTM



Multilayer architectures



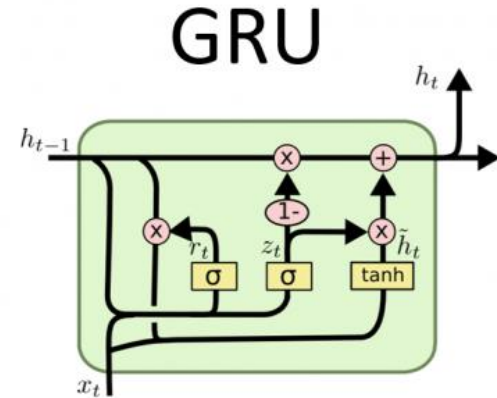
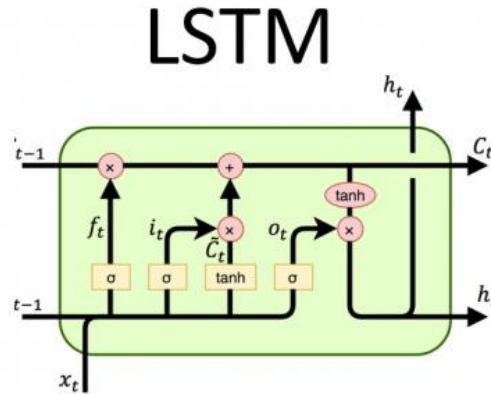
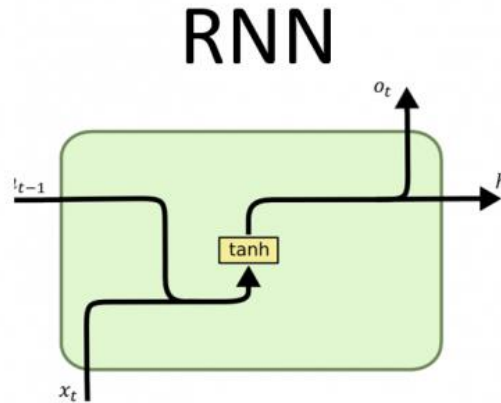
Other architectures: bidirectional LSTM



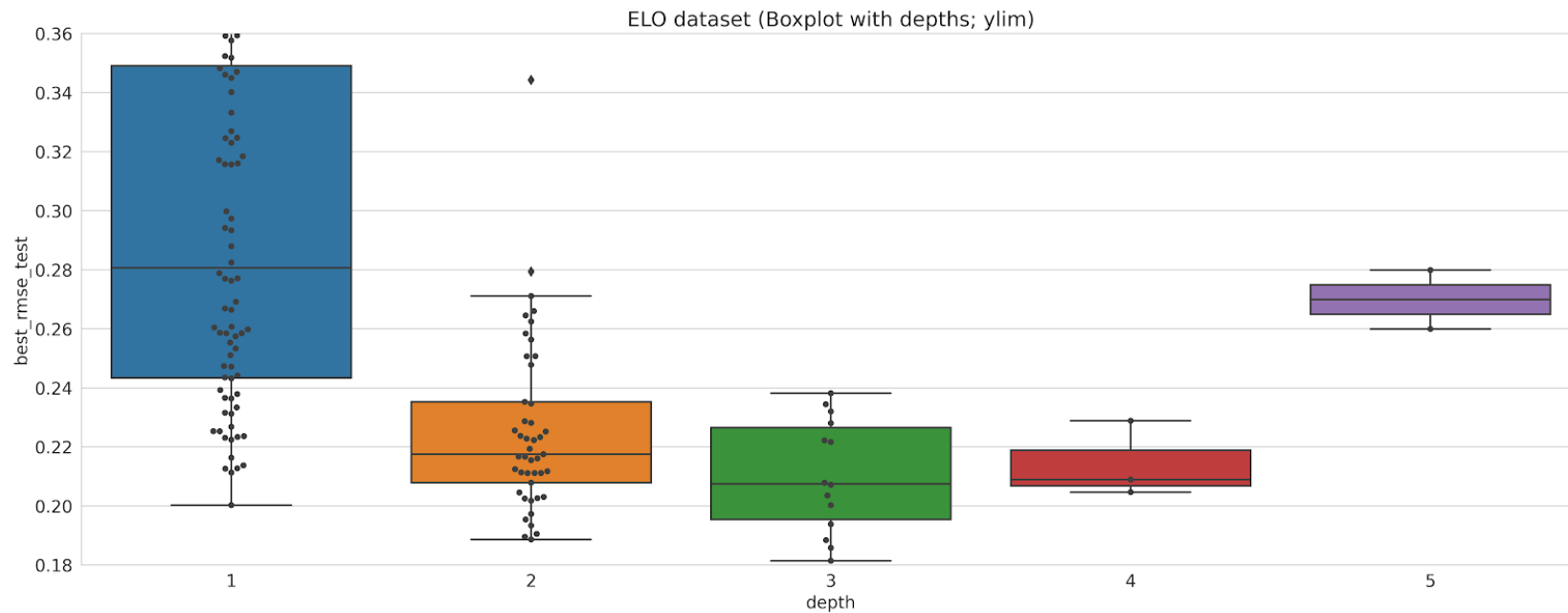
Other architectures: bidirectional LSTM

Bidirectional LSTM are useful when we benefit from the future data or can use it:

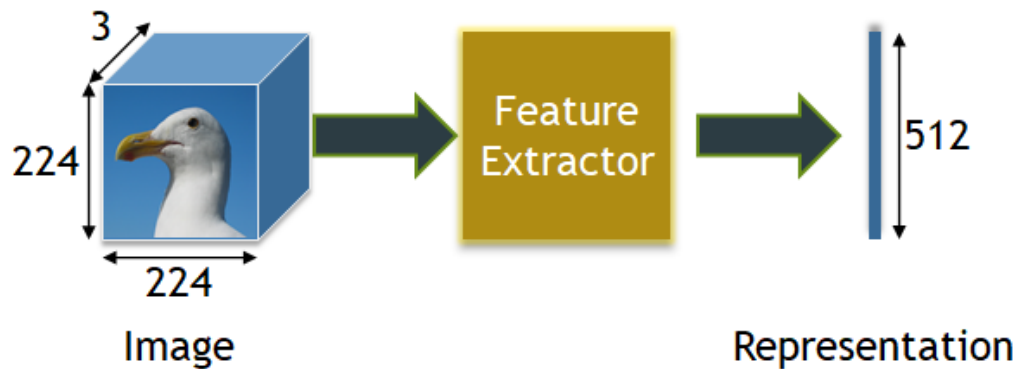
- Handwriting Recognition
- Speech Recognition
- Protein Structure Prediction (Bioinformatics)



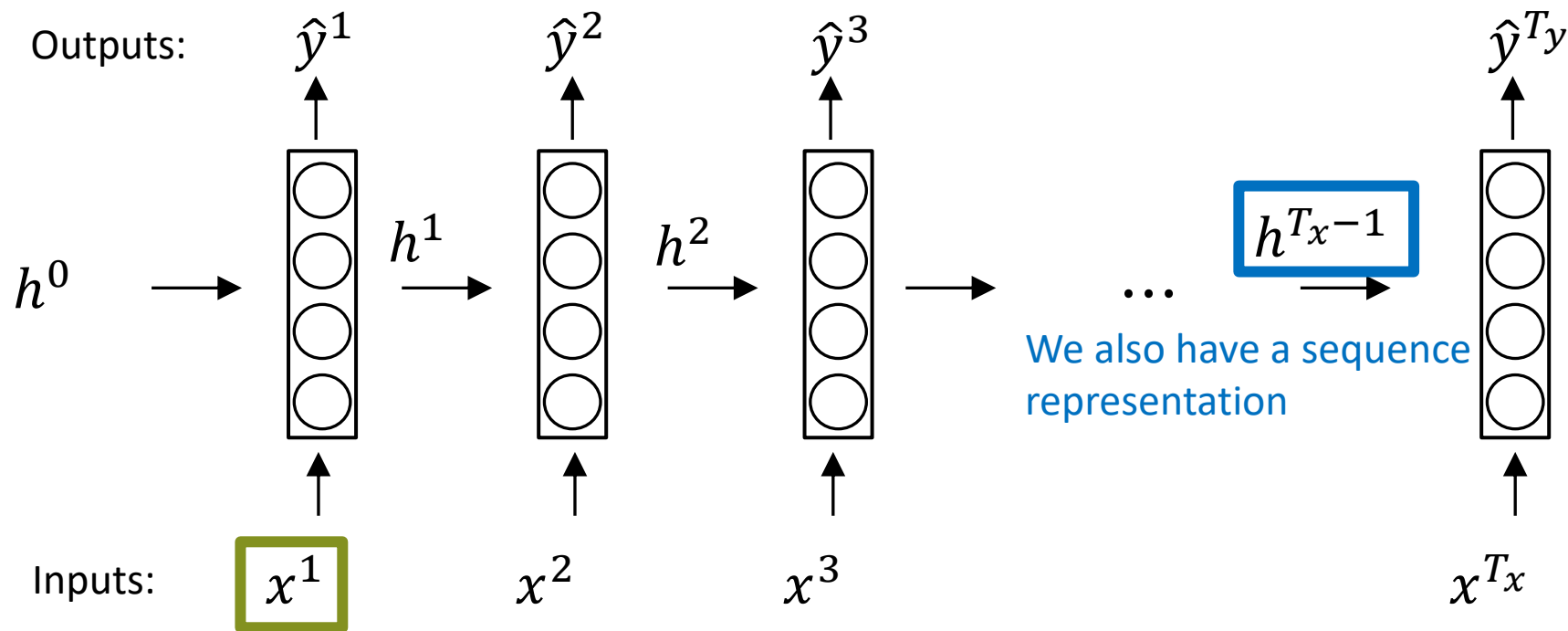
Multiple layers RNN performance



Representation learning is the core feature of Neural Networks



Representation learning is still here for Recurrent Neural Networks

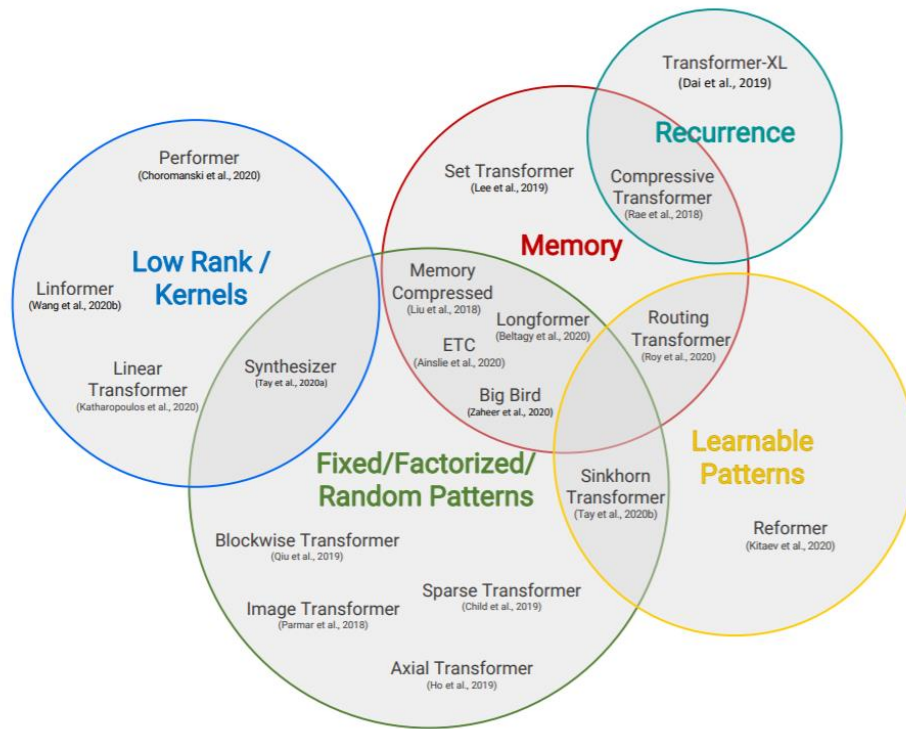


Most of the time we also learn representations of objects in an end2end manner with backpropagation

Transformers for long sequences

Transformer for long sequences

To work with sequences with significant length we should decrease memory consumption and computation complexity $O(n^2)$



BigBird approach

Memory requirements, n is the sequence length:

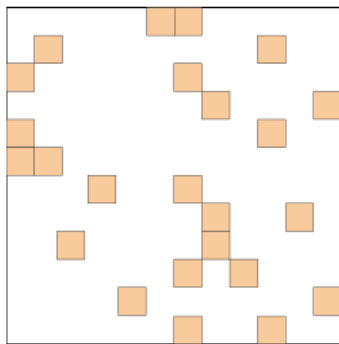
Random attention requires $O(r \cdot n)$

Sliding window requires $O(h \cdot n)$,
 h is the window size

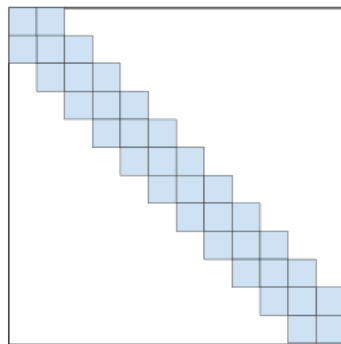
Global requires $O(g \cdot n)$, g is the global tokens number

BigBird combines 3 types of attention mechanism. All of them have linear complexity.

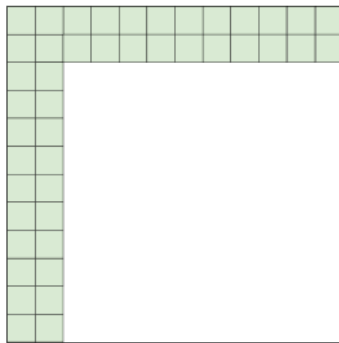
Total: $O((r + h + g) \cdot n)$



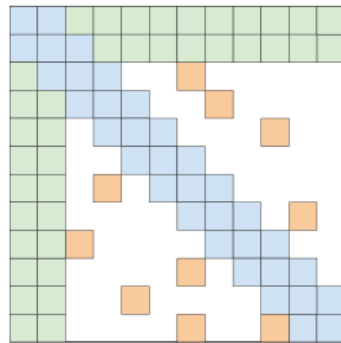
(a) Random attention



(b) Window attention



(c) Global Attention



(d) BIGBIRD